

## บทที่ 2

# ทฤษฎีพื้นฐานที่ใช้ในโครงงาน

### 2.1 ทฤษฎีการสแกนเพื่อตรวจสอบ

#### 2.1.1 Network Scan : ICMP Ping Sweeps

หนึ่งในเทคนิคพื้นฐานของ ICMP Network Scan ที่เป็นที่นิยมก็คือ การ ping ไปยังเครื่องเป้าหมายจำนวนมากพร้อมๆกัน ในลักษณะคล้ายกับการกวาดหรือกราดยิง ซึ่งเราเรียกว่าการทำ ping sweep เพื่อตรวจสอบว่าเครื่องปลายทางใดบ้างที่ยังเปิดทำงานอยู่ โดยปกติหากใช้คำสั่ง ping ธรรมดาจะมีการส่ง แพ็กเก็ต ICMP ECHO (Type 8) ออกไปยังเครื่องปลายทางและรอคอย ICMP ECHO\_REPLY (Type 0) ที่จะถูกส่งกลับมา ซึ่ง ping จะมีประโยชน์สำหรับการทดสอบว่าเครื่องปลายทางเปิดอยู่หรือไม่ โดยเทคนิคในการ ping sweep มีหลายเทคนิคแตกต่างกันไป เช่น การ ping ในลักษณะรอคอยการตอบสนองจากเครื่องทีละเครื่อง ก่อนจะเปลี่ยนไปทดสอบเครื่องอื่นๆ ถัดไปหรือจะเปลี่ยนเป็นการ ping ออกไปพร้อมๆกันในแบบขนานไปยังเครื่องปลายทางหลายๆเครื่อง ในลักษณะคล้าย Round Robin คือ การ ping ไปที่เครื่อง 1,2,3,...,n ถึงเครื่องสุดท้าย แล้ววนกลับมาส่งแพ็กเก็ตไปที่เครื่อง 1,2,3 ใหม่ไปเรื่อยๆ แล้ววนกลับมาอีก โดยไม่จำเป็นต้องหยุดรอจากตอบสนองจากเครื่องแรก ดังนั้น จะทำงานได้รวดเร็วกว่าคำสั่ง ping ธรรมดามาก นอกจากนี้ยังมีเทคนิคอื่นในการที่จะตรวจสอบว่าเครื่องปลายทางยังเปิดทำงานอยู่หรือไม่โดยการส่งแพ็กเก็ตของโปรโตคอล ICMP ใน Type อื่นๆด้วย เช่น การส่ง ICMP Type TIME STAMP request (13) เพื่อสอบถามเวลาของเครื่องปลายทาง (เพื่อดูว่าเครื่องๆนั้นอยู่ในโซนเวลา (time zone) แถบใด) แล้วรอคอย ICMP Type TIME STAMP Reply (14) หรือการส่ง ICMP INFO request (15) แล้วรอคอย ICMP Type INFO Reply (16) กลับมา

กล่าวโดยสรุป ICMP NETWORK SCAN นี้จะทำให้เราสามารถตรวจสอบได้ว่าเครื่องปลายทางใดบ้างที่เราสามารถติดต่อได้โดยตรง ด้วยการส่งและรอรับแพ็กเก็ต ICMP ใน TYPE ต่างๆ แต่วิธีการนี้จะเหมาะสำหรับเครื่องที่อยู่บนเน็ตเวิร์คขนาดเล็กถึงขนาดกลางเท่านั้น มันจะไม่มีประสิทธิภาพเพียงพอที่จะนำมาใช้ตรวจสอบเครื่องที่อยู่บนเน็ตเวิร์คขนาดใหญ่ได้ เนื่องจากการตรวจสอบเครื่องที่อยู่ในเน็ตเวิร์คในองค์กรขนาดใหญ่ อาจกินเวลานานหลายชั่วโมงกว่าจะทราบผลและถึงแม้จะอยู่บนเน็ตเวิร์คขนาดเล็กหรือขนาดกลางก็ตามแต่หากที่เน็ตเวิร์คเป้าหมายได้มีการบล็อกแพ็กเก็ต ICMP ไม่ว่าจะเป็นการบล็อกที่เราเตอร์ของเน็ตเวิร์คปลายทางหรือการใช้โปรแกรม Personal Firewall ที่เครื่องปลายทางก็ตามวิธีการนี้ก็จะมีประสิทธิภาพที่รวมทั้งในปัจจุบันวิธีการ Ping Sweep ถูกมองเป็นการโจมตีในรูปแบบของ DoS (Denial of

Service) เนื่องจาก Worm บางชนิดจะใช้วิธีการนี้ในการรวบรวมแบนวิดของเน็ตเวิร์คเป้าหมายด้วย จึงทำให้วิธีการนี้ไม่เหมาะสมในการตรวจสอบเน็ตเวิร์คในปัจจุบัน

### 2.1.2 Network Scan : Port Scanning

จากเหตุผลที่กล่าวมาทำให้จำเป็นต้องหาเทคนิคหรือเครื่องมืออื่นที่ใช้ตรวจสอบได้ว่าเครื่องปลายทางใดบ้างที่เข้าถึงได้และเปิดทำงานอยู่ แต่อย่างไรก็ตาม มันอาจไม่ถูกต้องและรวดเร็วเท่ากับตรวจสอบด้วย ping sweep เทคนิคที่ว่ามันก็คือ การสแกนพอร์ตนั่นเอง ซึ่งเป็นเทคนิคที่ใช้ถ้าเครื่องหรือเน็ตเวิร์คหรือปลายทางบล็อกแพ็กเก็ต ICMP ไว้ โดยการสแกนพอร์ตทั่วๆไปหรือสแกน

พอร์ตสามัญบนแต่ละเครื่องเราจะสามารถคาดการณ์ได้ว่าเครื่องไหนเปิดอยู่บ้างด้วยการส่ง TCP Packet หรือ UDP Packet ในรูปแบบต่างๆเพื่อคอนเน็กเข้าไปที่ TCP หรือ UDP port ของเครื่องปลายทางซึ่งทำให้สามารถตรวจสอบได้ว่าเครื่องปลายทางใดเชื่อมต่อกับเน็ตเวิร์คและเปิดทำงานอยู่ ยกตัวอย่างเช่นพอร์ต 80 เนื่องจากเป็นพอร์ตมาตรฐานที่ Access List หรือไฟร์วอลล์ของเราเตอร์หรือไฟร์วอลล์ส่วนใหญ่จะเปิดไว้ให้ผ่านเข้าไปยังเน็ตเวิร์คภายในและถึงแม้ เน็ตเวิร์คหรือเครื่องปลายทางจะบล็อกแพ็กเก็ต ICMP และพอร์ต 80 ไว้ก็อาจเปลี่ยนเป็นพอร์ตมาตรฐานที่มักพบบ่อย อย่างเช่น พอร์ต 25 (SMTP), 110 (POP), 113 (AUTH), 143 (IMAP) โดยถ้าหากเครื่องปลายทางนั้นได้เปิดพอร์ตที่สแกนไปนั้นก็จะมีการตอบกลับมาด้วย

นอกจากนั้นการสแกนพอร์ตยังสามารถใช้เพื่อค้นหาว่ามีพอร์ตอะไรบ้างที่ทำงานอยู่หรืออยู่ในสถานะ LISTENING การค้นหาพอร์ตที่เปิดอยู่เป็นเรื่องสำคัญทีเดียวในการตรวจสอบประเภทของระบบปฏิบัติการและแอปพลิเคชันที่ใช้งานอยู่ในระบบ เพราะ ระบบปฏิบัติการหรือเซิร์ฟเวอร์ที่รันอยู่อาจมีข้อบกพร่องบางอย่างเกี่ยวกับการเปิดพอร์ตที่อนุญาตให้ผู้ใช้ที่ไม่ได้ผ่านการตรวจสอบเข้าไปในระบบได้ หรือมีข้อบกพร่องเกี่ยวกับระบบการรักษาความปลอดภัยที่เป็นที่รู้จักกันดี โดยเฉพาะเซิร์ฟเวอร์บางเวอร์ชันที่ยังไม่สมบูรณ์นั่นเอง เครื่องมือและเทคนิคในการสแกนพอร์ตได้รับการพัฒนาอย่างต่อเนื่องมาหลาย สำหรับรูปแบบของการสแกนพอร์ตนั้นแบ่งเป็นรูปแบบที่รู้จักกันดีได้ดังนี้

**TCP connect scan** เป็นการคอนเน็กไปที่พอร์ตที่ต้องการบนเครื่องปลายทาง แล้วขอเปิดคอนเน็กชันของโพรโตคอล TCP ด้วยกลไกมาตรฐานที่เรียกว่า TCP three-way handshake (SYN, SYN/ACK และ ACK) โดยรูปแบบการทำงานคือ

1. เครื่องต้นทางส่งแพ็กเก็ต TCP ที่เซตแฟลก SYN เป็น 1 ไว้ไปที่เครื่องปลายทาง
2. เครื่องปลายทางส่งแพ็กเก็ต TCP SYN/ACK กลับมาที่เครื่องต้นทาง
3. เครื่องต้นทางส่งแพ็กเก็ต TCP ACK กลับไปเพื่อยืนยันว่าได้รับแพ็กเก็ตในขั้นที่สองแล้ว

**TCP SYN scan** เทคนิคนี้บางครั้งเรียกว่า half-open scanning สาเหตุเพราะว่า คอนเน็กชันที่สมบูรณ์ของโพรโตคอล TCP ยังไม่ได้ถูกเปิดขึ้น เพราะเมื่อได้รับแพ็กเก็ต TCP ที่เซตค่า

แฟล็ก SYN และ ACK ไว้เป็น 1 (TCP SYN/ACK) นั่นก็เพียงพอแล้วที่จะสรุปว่า พอร์ตดังกล่าวอยู่ในสถานะ LISTENING แต่ถ้าพอร์ตดังกล่าวไม่ได้เปิดอยู่ แพ็กเก็ต TCP ที่เซตค่าแฟล็ก RST และ ACK ไว้เป็น 1 (TCP RST/ACK) จะถูกส่งกลับมาแทน

**TCP FIN scan** เทคนิคนี้จะส่งแพ็กเก็ต TCP ที่เซตค่าแฟล็ก FIN เป็น 1 (TCP FIN) ไปยังพอร์ตที่ต้องการ ถ้าไดรเวอร์ของโพรโตคอล TCP/IP ที่เครื่องปลายทางได้ถูกพัฒนาขึ้นมาโดยมีฟีเจอร์ตามในมาตรฐาน RFC 793 เครื่องปลายทางจะส่งแพ็กเก็ต TCP RST ของทุกๆพอร์ตที่ปิดอยู่กลับมาให้ เทคนิคนี้มักใช้ได้กับเครื่องปลายทางที่ใช้ระบบปฏิบัติการยูนิกซ์หรือลินุกซ์

**TCP Xmas Tree scan** เทคนิคนี้จะส่งแพ็กเก็ต TCP ที่เซตแฟล็ก FIN ,URG และ PUSH ไปยังพอร์ตเป้าหมายที่เครื่องปลายทาง และอาศัยมาตรฐาน RFC 793 อีกเช่นกัน เครื่องปลายทางจะส่งแพ็กเก็ต TCP RST ของทุกพอร์ตที่ปิดอยู่กลับมาให้

**TCP Null scan** เทคนิคนี้จะส่งแพ็กเก็ต TCP ออกไปโดยเซตค่าของทุกๆแฟล็กให้เป็น 0 ทั้งหมด และ อาศัยมาตรฐาน RFC 793 อีกเช่นกัน เครื่องปลายทางจะส่งแพ็กเก็ต TCP RSTของทุกๆพอร์ตที่ปิดอยู่กลับมาให้

**TCP ACK scan** เทคนิคนี้จะถูกใช้เพื่อค้นหา “rule” และ “policy” ต่างๆที่เซตไว้ที่ไฟร์วอลล์เพื่อตรวจสอบว่าไฟร์วอลล์นั้นๆทำหน้าที่แค่เพียงกรองแพ็กเก็ตอย่างง่ายๆ หรือเป็นไฟร์วอลล์ที่มีความฉลาดพอสมควรและใช้เทคนิคการกรองแพ็กเก็ตขั้นสูง

**TCP Window scan** เทคนิคนี้จะตรวจสอบพอร์ตที่เปิดอยู่ รวมทั้งตรวจสอบว่า พอร์ตใดบ้างที่ถูกฟิลเตอร์เอาไว้ไม่ให้ผ่านเข้าไปถึง และพอร์ตหมายเลขใดได้รับการอนุญาตไว้บ้าง โดยอาศัยช่องโหว่จากความผิดปกติบางอย่างในการแจ้งค่าของ TCP Window Size ของโพรโตคอลTCP/IP

**TCP RPC scan** เทคนิคนี้ใช้งานได้เฉพาะกับเครื่องปลายทางที่รันยูนิกซ์เท่านั้น มันถูกใช้เพื่อตรวจสอบว่ามีเซอร์วิสใดทำงานอยู่บนเซอร์วิส RPC บ้าง รวมทั้งตรวจสอบเวอร์ชันของเซอร์วิสนั้นและโปรแกรมอื่นที่เกี่ยวข้อง

**UDP scan** เทคนิคนี้จะส่งแพ็กเก็ตของโพรโตคอล UDP ไปยังพอร์ตเป้าหมาย ถ้าเครื่องปลายทางตอบกลับมาด้วยแพ็กเก็ต ICMP type PORT UNREACHABLE นั้นหมายความว่า พอร์ตนั้นปิดอยู่ในทางตรงกันข้าม ถ้าเราไม่ได้รับแพ็กเก็ต ICMP type ดังกล่าว เราสามารถสรุปได้ว่าพอร์ตนั้นเปิดอยู่ เนื่องจากโพรโตคอล UDP เป็นโพรโตคอลลักษณะconnectionless คือไม่รับรองว่าแพ็กเก็ตที่ส่งไปจะถึงเครื่องปลายทางครบถ้วนหรือไม่ ดังนั้นความถูกต้องของผลลัพธ์ที่ได้จากเทคนิคนี้ก็อาจขึ้นกับปัจจัยอื่นๆ ด้วยเช่น ปริมาณทราฟฟิกในเน็ตเวิร์คและทรัพยากรบนเครื่องปลายทาง นอกจากนั้นมันยังเป็นเทคนิคที่ค่อนข้างช้าอีกด้วย

สำหรับไดรเวอร์ของโพรโตคอล TCP/IP ของระบบปฏิบัติการบางตัวอาจส่งแพ็กเก็ต TCP RST กลับไปยังเครื่องต้นทางตลอดไม่ว่าพอร์ตๆ นั้นจะเปิดหรือปิดอยู่ ดังนั้นจึงเป็นไปได้ว่าผลลัพธ์ที่ได้ อาจแตกต่างกันไปไม่แน่นอน แต่อย่างไรก็ดี การตรวจสอบเครื่องปลายทางด้วยการ Scan Port นั้นนับว่ามีประสิทธิภาพพอสมควร

### 2.1.3 การตรวจหาประเภทของระบบปฏิบัติการ : Impossible Flags และ OS Fingerprint

การตรวจสอบว่าเครื่องปลายทางเป็นระบบปฏิบัติการอะไร เวอร์ชันไหนเป็นการใช้ข้อบกพร่องของโปรโตคอลที่มีได้มีการกำหนดชัดเจนครอบคลุมทุกเงื่อนไขของการสื่อสารข้อมูล ดังนั้นเมื่อเหตุการณ์ดังกล่าวเกิดขึ้น จึงไม่มีมาตรฐานที่ทุกคนต้องยึดร่วมกัน การตอบสนองของระบบปฏิบัติการแต่ละชนิดจึงแตกต่างกันออกไปขึ้นอยู่กับผู้ผลิตระบบปฏิบัติการนั้นจะอิมพลีเมนต์ TCP/IP อย่างไร และแน่นอนว่าระบบปฏิบัติการแต่ละรุ่นก็มีลักษณะการตอบสนองแตกต่างกันออกไป การสแกนด้วยเทคนิคนี้ส่วนใหญ่จะกระทำบนโปรโตคอล TCP เนื่องจากมีส่วนที่ไม่ได้กำหนดรูปแบบทั้งหมดไว้ในโปรโตคอลอยู่นั่นคือ TCP Flag ซึ่งเป็นส่วนสำคัญที่ใช้ในการควบคุมการสื่อสารของ TCP แต่ TCP Flag เหล่านั้นก็ไม่ได้ถูกนำไปใช้งานทุกๆ เงื่อนไขจึงมี Flag บางเงื่อนไขซึ่งไม่มีโอกาสเกิดขึ้นได้จริงในการทำงานปกติ (Impossible TCP Flags) ซึ่งระบบปฏิบัติการแต่ละชนิดแต่ละเวอร์ชันก็จะตอบสนองต่อแพ็กเก็ตลักษณะนี้แตกต่างกันออกไปตามเหตุผลที่กล่าวไปแล้วและเมื่อรวบรวมผลลัพธ์ที่ตอบมาจากระบบปฏิบัติการแต่ละชนิดด้วย Impossible Flag หลายๆ แบบแล้วจะพบว่ารูปแบบของข้อมูลเหล่านี้สามารถบ่งบอกถึงระบบปฏิบัติการได้ซึ่งเรียกรูปแบบของการตอบสนองของ Impossible Flags นี้ว่า OS Fingerprint ดังนั้นความแม่นยำของการทดสอบหาระบบปฏิบัติการด้วยวิธีนี้จะขึ้นอยู่กับจำนวนของแพ็กเก็ตที่ใช้ทดสอบ ยิ่งมีการทดสอบหลายแพ็กเก็ตผลการตอบรับมากก็จะยิ่งชี้ไปยังระบบปฏิบัติการรุ่นใดรุ่นหนึ่งได้อย่างแม่นยำมากขึ้นส่วนอีกปัจจัยหนึ่งคือความทันสมัยของการปรับปรุงฐานข้อมูลลายนิ้วมือให้ทันต่อระบบปฏิบัติการเวอร์ชันใหม่ๆ อยู่เสมอ

### 2.1.4 การตรวจหาประเภทของระบบปฏิบัติการ : Active Stack Fingerprinting

Stack fingerprinting เป็นเทคโนโลยีที่ช่วยทำให้มั่นใจได้ว่าประเภทของระบบปฏิบัติการที่ค้นหาได้นั้นเป็นระบบปฏิบัติการที่ถูกต้อง มีเปอร์เซ็นต์ความน่าเชื่อถือสูง โดยอาศัยหลักการที่ว่า ผู้ผลิตระบบปฏิบัติการแต่ละรายมักพัฒนาไดรเวอร์ของโปรโตคอล TCP/IP และเซอร์วิสที่ทำงานบนโปรโตคอล TCP/IP ให้มีเอกลักษณ์เฉพาะตัวเป็นของตัวเอง ซึ่งมักแตกต่างกับของระบบปฏิบัติการอื่น ดังนั้น โดยการตรวจวัดความแตกต่างเหล่านี้ เราจะสามารถเริ่มคาดเดาได้อย่างมีเหตุผล แต่เพื่อให้มีความน่าเชื่อถือสูงสุด Stack fingerprinting จำเป็นต้องอาศัยการคอนเน็กไปยังพอร์ตที่เปิดอยู่อย่างน้อยหนึ่งพอร์ต แต่การทำงานของโครงการนี้ สามารถคาดเดาได้อย่างมีเหตุผลถึงแม้ว่าจะไม่ได้คอนเน็กไปที่พอร์ตใดเลย แต่ผลที่ได้ก็อาจยังไม่น่าเชื่อถือนักโดยอาศัยวิธีการดังนี้

**Fin probe** ใช้การส่งแพ็กเก็ตของโปรโตคอล TCP โดยเซตแฟล็ก Fin ให้เป็น 1 ไว้ตามมาตรฐาน RFC 793 ระบุไว้ว่า พฤติกรรมที่ถูกต้องจะต้องไม่มีการส่งแพ็กเก็ตอะไรตอบสนองกลับไป แต่ทว่า ไดรเวอร์ของโปรโตคอล TCP/IP ในระบบปฏิบัติการบางตัว อย่างเช่น วินโดวส์เอ็นที จะตอบสนองกลับมาด้วยแพ็กเก็ตของโปรโตคอล TCP ที่เซตแฟล็ก Fin และ ACK ให้เป็น 1 ซึ่งเครื่องมือส่วนมากได้นำเทคนิคนี้ไปใช้กัน

**Bogus Flag probe** ใช้การส่งแพ็กเก็ตของโพรโทคอล TCP โดยเซตแฟลก SYN ให้เป็น 1 พร้อมทั้งเซตบิตตำแหน่งที่ยังไม่ได้ใช้งานให้เป็น 1 ด้วย บางระบบปฏิบัติการเช่น ลินุกซ์ จะตอบสนองกลับมาด้วยการเซตแฟลกบางตัวในแพ็กเก็ต ว่ามักเป็นค่าอะไร อย่างไรก็ตามบางระบบปฏิบัติการจะทำการ reset connection เมื่อได้รับแพ็กเก็ตนี้

**TCP Initial Sequence Number (ISN) sampling** ใช้วิธีการตรวจหาแพทเทิร์นของ Sequence number ที่อยู่ในส่วนหัวของแพ็กเก็ตที่ได้รับการตอบสนองต่อการร้องขอการเชื่อมต่อว่าเป็นค่าอะไร ซึ่งสามารถแบ่งได้หลายกลุ่มเช่น ถ้าเป็นแบบ traditional 64K จะพบใน UNIX รุ่นเก่า, แบบ Random increments จะพบใน Solaris, IRIX, FreeBSD, Digital UNIX, Cray ในเวอร์ชันใหม่, แบบ True "random" จะพบใน Linux 2.0.\*, OpenVMS, AIX เวอร์ชันใหม่, ส่วน Windows จะเป็นแบบ "time dependent" model คือค่าของ ISN จะเพิ่มขึ้นจำนวนหนึ่งที่กำหนดไว้ในแต่ละช่วงเวลา, และในบางครั้งจะใช้ค่า ISN ค่าเดียวไม่เปลี่ยนแปลง

**"Don't fragment bit" monitoring** บางระบบปฏิบัติการได้เซตบิต "Don't fragment bit" ไว้เพื่อเพิ่มความเร็วในการส่งข้อมูล ให้มอนิเตอร์บิตนี้เพื่อตรวจดูว่าระบบปฏิบัติการไหนเซตบิตนี้บ้าง

**TCP initial window size** ใช้วิธีดูขนาดของ TCP window เพราะบางระบบปฏิบัติการจะมีการลือค่าไว้เลยว่าขนาดของ TCP window เป็นเท่าไร (เช่น AIX เป็นเพียงระบบปฏิบัติการเดียวที่มีขนาดของ window เท่ากับ 0x3F25 ส่วน window NT มีขนาด 0x402E) ค่านี้มักเป็นค่าเฉพาะตัวของแต่ละระบบปฏิบัติการด้วย จึงยังทำให้ผลที่ได้มีความถูกต้องมากขึ้น

**ACK value** ระบบปฏิบัติการแต่ละระบบมักเซตค่าในฟิลด์ ACK ไม่เหมือนกัน บางระบบเซตค่าฟิลด์ ACK ให้สอดคล้องตามค่าของฟิลด์ SYN ที่เซตไว้ในฝั่งผู้ส่ง บางระบบจะเซตค่าฟิลด์ ACK ให้บวกจากค่าของฟิลด์ SYN ไปอีกหนึ่ง

**ICMP error message quenching** บางระบบปฏิบัติการอาจปฏิบัติตามมาตรฐาน RFC 1812 และมีการจำกัดอัตราการส่งข้อความแจ้งข้อผิดพลาดดังนั้น (ใน Linux kernel จะจำกัดการส่งข้อความแจ้งข้อผิดพลาดไปยังปลายทางที่อัตรา 80 แพ็กเก็ตต่อ  $4 \pm 0.25$  วินาที) โดยการส่งแพ็กเก็ตของโพรโทคอล UDP ไปยังหมายเลขพอร์ตสูงๆ แล้วค่อยนับจำนวนของ ICMP type PORTUNREACHABLE Message ที่ตอบกลับมาภายในช่วงเวลาที่กำหนด

**ICMP message quoting** เมื่อพบข้อผิดพลาดเกี่ยวกับโพรโทคอล TCP/IP ระบบปฏิบัติการแต่ละประเภทจะให้ข้อมูลและสาเหตุต่างๆมาในแพ็กเก็ตของ ICMP ไม่เท่ากัน บางระบบจะให้รายละเอียดมากบางระบบจะให้รายละเอียดน้อย (ใน Solaris จะส่งกลับมา a bit more และใน Linux จะส่งกลับมา even more than that) ดังนั้นโดยการสำรวจข้อมูลที่ได้มานี้ เราพอจะใช้ในการคาดเดาประเภทของระบบปฏิบัติการได้

**ICMP error message-echoing integrity** บางระบบปฏิบัติการอาจดัดแปลงส่วนหัวของแพ็กเก็ต IP เมื่อมีการส่ง ICMP error message ด้วยการตรวจสอบลักษณะของการดัดแปลงดังกล่าวนี้คุณสามารถนำมาใช้ในการคาดเดาได้

**Type of service (TOS)** ให้คุณสำรวจฟิลด์ที่ชื่อ Type of service ที่อยู่ในแพ็กเก็ต ICMP ประเภท port unreachable ซึ่งโดยปกติหลายระบบปฏิบัติการจะใช้ค่าศูนย์ แต่บางระบบอาจใช้ค่าอื่น เช่น Linux ใช้ 0xC0

**Fragmentation handling** แต่ละระบบปฏิบัติการจะจัดการกับแพ็กเก็ตที่ถูกแฟรกเมนต์หรือหั่นซอยไม่เหมือนกัน เมื่อมีการประกอบแพ็กเก็ตย่อยขึ้นมาเป็นแพ็กเก็ตที่สมบูรณ์ บางระบบจะเขียนทับแพ็กเก็ตย่อยอันเก่าด้วยแพ็กเก็ตย่อยอันใหม่หรือบางระบบก็ทำในทางตรงกันข้าม โดยการสังเกตพฤติกรรมดังนี้ เราพอจะใช้ในการคาดเดาประเภทของระบบปฏิบัติการได้เช่นเดียวกัน

**TCP options** ได้ถูกกำหนดไว้ในมาตรฐาน RFC 793 และได้รับการปรับปรุงในมาตรฐาน RFC 1323 ในปัจจุบัน ผู้ผลิตหลายรายได้มีการอิมพลีเมนต์ออปชันพิเศษที่อยู่ใน RFC 1323 ไว้ในระบบปฏิบัติการของตน ดังนั้น ด้วยการส่งแพ็กเก็ตที่เซตออปชันหลายๆออปชันไปทดสอบอย่างเช่น no operation, maximum segment size, window scale factor และ timestamps มันเป็นไปได้ที่เราจะตั้งสมมติฐานเกี่ยวกับระบบปฏิบัติการที่รันที่เครื่องเป้าหมาย

#### ยกตัวอย่าง

Window Scale=10; NOP; Max Segment Size = 265; Timestamp; End of Ops;

บางระบบปฏิบัติการ เช่น FreeBSD จะ support ทุกออปชันข้างต้นขณะที่ Linux 2.0.X จะ support บางออปชัน Linux 2.1.x support ทุกออปชันแม้ว่าหลายระบบปฏิบัติการจะ support ออปชันเดียวกันแต่บางที่เราก็สามารถแยกแยะได้โดยจากออปชัน the\_values\_of เช่น ถ้าส่งค่า MSS เล็กๆไปที่ Linux โดยทั่วไปแล้วมันจะส่ง MSS echo กลับไป ส่วนระบบปฏิบัติการอื่นจะส่งค่าที่แตกต่างกันกลับไป แต่ถ้ายังได้ผลลัพธ์เหมือนกันอีกก็สังเกตลำดับของออปชันที่ได้รับมา เช่น Solaris จะเป็น 'NNTNWME' ซึ่งหมายความว่า <no op><no op><timestamp><no op><window scale><echoed MSS> ขณะที่ Linux 2.1.122 จะเป็น MENNTNW ซึ่งจะเห็นได้ว่าเป็นออปชันเดียวกัน ส่ง MSS echo เหมือนกัน แต่ลำดับที่ส่งมาไม่เหมือนกัน

**SYN Flood Resistance** บางระบบปฏิบัติการจะไม่ยอมรับการ connection ครั้งใหม่ ถ้ามีการส่งแพ็กเก็ต SYN ไปมากเกินไป ซึ่งหลายระบบปฏิบัตินั้นสามารถที่จะจัดการกับแพ็กเก็ตที่ส่งเข้ามาได้ไม่เกินครั้งละ 8 แพ็กเก็ต แต่ใน kernel ของ Linux รุ่นใหม่มีวิธีในการป้องกันปัญหานี้ เช่น 64 การใช้ SYN cookies ดังนั้นเราก็สามารถรู้ได้ว่าเป็นระบบ ปฏิบัติการอะไรโดยการส่งแพ็กเก็ตไป 8 แพ็กเก็ตไปยังพอร์ตที่เปิดอยู่แล้วดูว่าสามารถ connect ไปยังพอร์ตนั้นได้หรือไม่

## 2.2 ทฤษฎี และหลักการในส่วนของ SNMP และ RMON

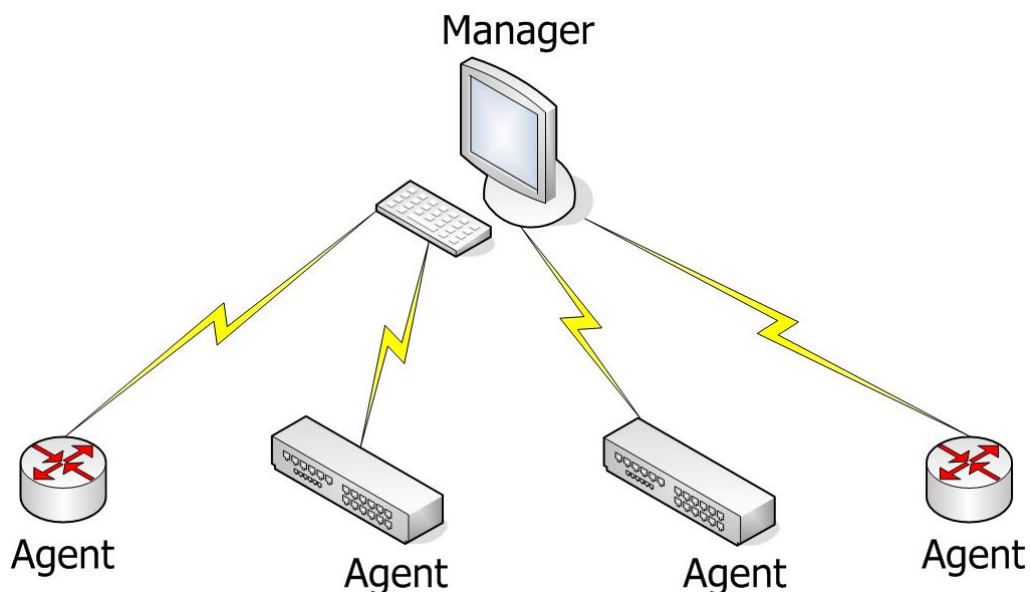
SNMP เป็นโปรโตคอลในระดับประยุกต์ (Application Layer) ที่กำหนดรูปแบบ และกรรมวิธีการจัดการเครือข่าย โดยมีสถานีจัดการเครือข่ายส่วนกลางทำหน้าที่ดูแล ตรวจสอบ และควบคุมการทำงานของอุปกรณ์เครือข่าย

### 2.2.1 พื้นฐานการบริหารเครือข่าย

การบริหารเครือข่าย คือ การตรวจ ควบคุม และวางแผนการใช้ทรัพยากรระบบเพื่อให้เครือข่ายทำงานได้อย่างมีประสิทธิภาพ และ สามารถตรวจหาจุดบกพร่องที่เกิดขึ้นเพื่อแก้ไขปัญหาลงได้อย่างรวดเร็ว

ในระบบเครือข่ายใดๆที่ ต้องใช้การจัดการเครือข่าย TCP/IP จะต้องประกอบด้วย 4 ส่วน (Model) ด้วยกัน คือ

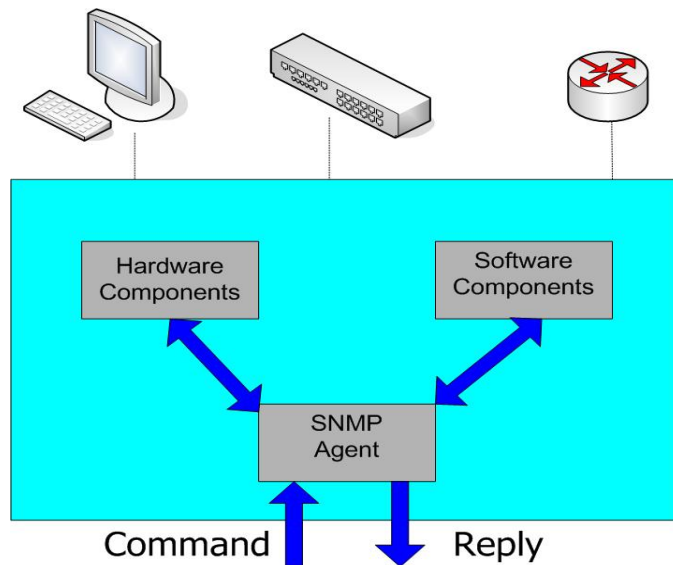
Management station : เป็นสถานีการจัดการเครือข่ายส่วนกลางซึ่งทำหน้าที่ดูแล ตรวจสอบ และ ควบคุม การทำงานของอุปกรณ์ในเครือข่าย ดังรูปที่ 2.1



รูปที่2.1 แสดงองค์ประกอบในระบบจัดการเครือข่าย

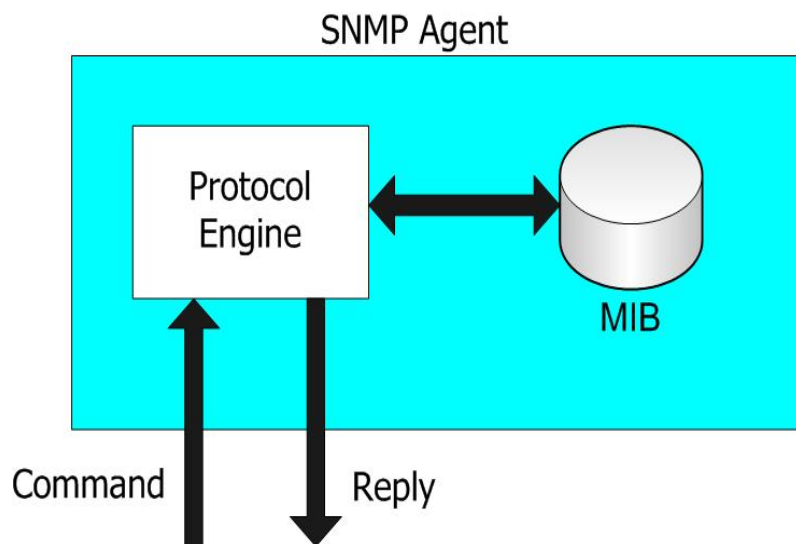
Management agent : เป็นสมาชิกในระบบการจัดการเครือข่าย ซึ่งมีฟังก์ชันที่ให้ตรวจสอบ และปรับเปลี่ยนการทำงานได้ อาจจะเป็น โฮสต์, บริดจ์, สวิตช์, เราท์เตอร์, ฮับ หรืออุปกรณ์อื่นๆก็ได้ที่สามารถส่งข้อมูลสถานะของมันออกไปยังระบบได้ จะเป็นอุปกรณ์ ฮาร์ดแวร์ หรือ ซอฟต์แวร์ ก็ได้ แต่ต้องมี เอสเอ็นเอ็มพีเอเจนต์ อยู่ในตัวด้วย

โดยเอเจนต์จะนำการทำงานของซอฟต์แวร์ หรือฮาร์ดแวร์ เมื่อสถานีจัดการเครือข่าย ร้องขอข้อมูล และปรับเปลี่ยนการของซอฟต์แวร์ หรือฮาร์ดแวร์ เมื่อสถานีจัดการเครือข่ายสั่งงาน โดยมีการยืนยันสิทธิในรูปรหัสผ่านว่ามีอำนาจหน้าที่ในการร้องขอ และปรับค่าได้ ดังรูป 2.2



รูปที่ 2.2 เอสเอ็นเอ็มพีเอเจนต์

เอเจนต์ประกอบด้วยส่วนสำคัญ 2 ส่วนด้วยกัน คือ โปรโตคอลเอ็นจิน (Protocol engine) และฐานข้อมูลสารสนเทศการจัดการ (Management information base) ดังรูป 2.3



รูปที่ 2.3 โครงสร้างของเอเจนต์

โปรโตคอลเอ็นจินทำหน้าที่ประมวลคำสั่งที่มาจากสถานีจัดการเครือข่ายได้แก่ รับคำสั่ง ถอดรหัสคำสั่ง ทำงานตามคำสั่ง และส่งผลตอบกลับ ฐานข้อมูลสารสนเทศการจัดการเป็นส่วนที่เก็บตัวแปร และค่ากำหนดการทำงานประจำอุปกรณ์

Management information base (MIB) : ฐานข้อมูลสารสนเทศการจัดการเป็นส่วนที่เก็บตัวแปร และค่ากำหนดการทำงานประจำอุปกรณ์



Network management protocol : ทำหน้าที่ติดต่อระหว่างสถานีจัดการ กับเอเจนต์ มีรูปแบบในการติดต่อ หลายรูปแบบด้วยกันตามวัตถุประสงค์ในการติดต่อ โดยการจัดการเครือข่ายใน TCP/IP จะอาศัยรูปแบบการจัดการมาตรฐานตามข้อกำหนดของโปรโตคอล SNMP ซึ่งเป็นโปรโตคอลประยุกต์ที่กำหนดรูปแบบ และกรรมวิธีจัดการเครือข่าย

### 2.2.2 Communities และ Community Names

การบริหารเครือข่ายจะถือได้ว่าเป็นการทำงานในลักษณะระบบกระจาย (Distributed application)รูปแบบหนึ่ง ซึ่งจะเห็นได้ว่าความสัมพันธ์ระหว่างสถานีจัดการเครือข่าย (Network Management Station : NMS) กับเอเจนต์ (Agent) จะเป็นในรูปแบบ many-to-many คือ สถานีจัดการเครือข่ายจะบริหารจัดการเอเจนต์ได้หลายเครื่อง และในขณะเดียวกันเอเจนต์ก็สามารถถูกบริหารควบคุมจากสถานีจัดการเครือข่ายหลายเครื่องเช่นกัน

จากความสัมพันธ์ดังกล่าวจึงจำเป็นที่เอเจนต์แต่ละเครื่องจำเป็นที่จะต้องมีการควบคุมความปลอดภัยในการใช้งานฐานข้อมูลสารสนเทศการจัดการ (Management Information Base : MIB) ของตนเองโดยจะมีมุมมองทางด้านความปลอดภัย 3 ประการได้แก่

1. การพิสูจน์ตัวตน (Authentication service) จะเป็นการจำกัดให้เฉพาะสถานีจัดการเครือข่ายที่จะเข้ามาบริการควบคุม
2. นโยบายการเข้าถึง (Access policy) จะมีการกำหนดระดับการอนุญาตการเข้าถึงฐานข้อมูลสารสนเทศการจัดการให้แต่ละสถานีจัดการเครือข่ายไม่เท่ากันในแต่ละเครื่อง
3. การให้บริการ Proxy (Proxy service) เอเจนต์อาจทำหน้าที่เป็น Proxy ให้กับเอเจนต์เครื่องอื่นซึ่งจะรวมถึงการพิสูจน์ตัวตนและนโยบายการเข้าถึงของเอเจนต์ตัวอื่นที่อยู่ในระบบ Proxy

เอสเอ็นเอ็มพี (SNMP) ได้มีการกำหนดการทำงานเพื่อสนับสนุนมุมมองทางด้านความปลอดภัยดังกล่าวในรูปแบบของ SNMP Community โดยการทำงานคือ เอเจนต์แต่ละเครื่องจะมีการสร้าง Community name เพื่อกำหนดให้กับสถานีจัดการเครือข่าย โดยในหนึ่ง Community name จะสามารถมีสถานีจัดการเครือข่ายมากกว่าหนึ่งตัว

เนื่องจาก Community name จะถูกกำหนดในแต่ละเอเจนต์จึงอาจเป็นไปได้ว่ามีการตั้งชื่อ Community name ซ้ำกันในแต่ละเอเจนต์ แต่สถานีจัดการเครือข่ายจะสามารถแยกความแตกต่างของ Community ที่มีชื่อซ้ำกันเหล่านี้เองได้ถ้าอยู่ในคนละเอเจนต์กัน ดังนั้นจึงจำเป็นที่ว่าสถานีจัดการเครือข่ายจะต้องเก็บข้อมูลของ Community name และข้อมูลที่เกี่ยวข้องของแต่ละเอเจนต์เพื่อใช้ในการบริหารควบคุม

#### การพิสูจน์ตัวตน (Authentication service)

การพิสูจน์ตัวตนมีไว้เพื่อให้แน่ใจการติดต่อนั้นเป็นของแท้ ในกรณีของเอสเอ็นเอ็มพีการพิสูจน์ตัวตนจะมีไว้เพื่อให้แน่ใจว่า message ที่ได้รับมานั้นเป็นข้อความที่แท้จริง โดยในทุกๆ

message ของเอสเอ็นเอ็มพีจะมีการระบุ Community name ซึ่งจะมีหน้าที่เหมือนกับรหัสผ่าน (Password) ในการพิสูจน์ตัวตน นอกจากนี้ยังอาจจะมีการเข้ารหัส (Encryption) เพื่อเพิ่มความปลอดภัยในการพิสูจน์ตัวตนมากยิ่งขึ้น

### นโยบายการเข้าถึง (Access policy)

การควบคุมการเข้าถึงในเอสเอ็นเอ็มพีจะประกอบด้วย 2 องค์ประกอบหลักที่เกี่ยวข้องคือ

1. SNMP MIB view: คือกลุ่มของอ็อบเจกต์ในฐานข้อมูลสารสนเทศการจัดการที่ตั้งขึ้นโดย ในแต่ละกลุ่มอาจจะประกอบด้วยหลาย Sub tree ในฐานข้อมูลสารสนเทศการจัดการได้
2. SNMP access mode: คือรูปแบบของการเข้าถึงได้แก่ READ-ONLY และ READWRITE

ในเอเจนต์จะมีการกำหนด Access mode ให้แต่ละ MIB view ซึ่ง Access mode จะมีผลกับทุกๆ Object ที่อยู่ในกลุ่มของ MIB view โดยทั้ง Access mode และ MIB view จะถูกรวบรวมกันว่า SNMP community profile ซึ่งจะถูกกำหนดในแต่ละ Community

ใน Access mode ของเอสเอ็นเอ็มพีจะมีการประนีประนอมต่อรองระดับการเข้าถึงกับระดับของการเข้าถึงของฐานข้อมูลสารสนเทศการจัดการ (MIB Access Category) ดังตารางที่ 2.1 (Access mode ของเอสเอ็นเอ็มพีกับระดับของการเข้าถึงของฐานข้อมูลสารสนเทศการจัดการเป็นนัยยะส่วนกัน) และจะเรียกรวม SNMP community และ SNMP community profile ว่า SNMP access policy

| MIB Access Category | SNMP Access Mode                                                           |                                                                                                              |
|---------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
|                     | READ-ONLY                                                                  | READ-WRITE                                                                                                   |
| read-only           | สามารถใช้คำสั่ง get และ trap ได้                                           |                                                                                                              |
| read-write          | สามารถใช้คำสั่ง get และ trap ได้                                           | สามารถใช้คำสั่ง get, set และ trap ได้                                                                        |
| write-only          | สามารถใช้คำสั่ง get และ trap แต่ต้องเป็นค่าที่เป็น implementation-specific | สามารถใช้คำสั่ง get, set และ trap ได้แต่ต้องเป็นค่าที่เป็น implementation-specific สำหรับคำสั่ง get และ trap |
| not accessible      | ไม่สามารถเข้าถึงได้                                                        |                                                                                                              |

ตารางที่ 2.1 ความสัมพันธ์ระหว่าง MIB Access Category และ SNMP Access Mode

### การให้บริการ Proxy (Proxy service)

แนวคิดของ Community จะสามารถสนับสนุนการทำ Proxy ได้ โดยเอเจนต์จะสามารถทำหน้าที่เป็นตัวแทนในการติดต่อกับสถานีจัดการเครือข่ายให้กับเอเจนต์หรืออุปกรณ์อื่นได้ ในกรณีที่อุปกรณ์เครื่องนั้นไม่สนับสนุน TCP/IP และเอสเอ็นเอ็มพีหรือในกรณีที่ต้องการลดปริมาณ

การติดต่อระหว่างอุปกรณ์นั้นกับสถานีจัดการเครือข่าย โดยเอเจนต์ที่เป็น Proxy จะสนับสนุน SNMP access policy ของเอเจนต์ที่อยู่ในระบบ Proxy ด้วย

### 2.2.3 MIB (Management Information Base)

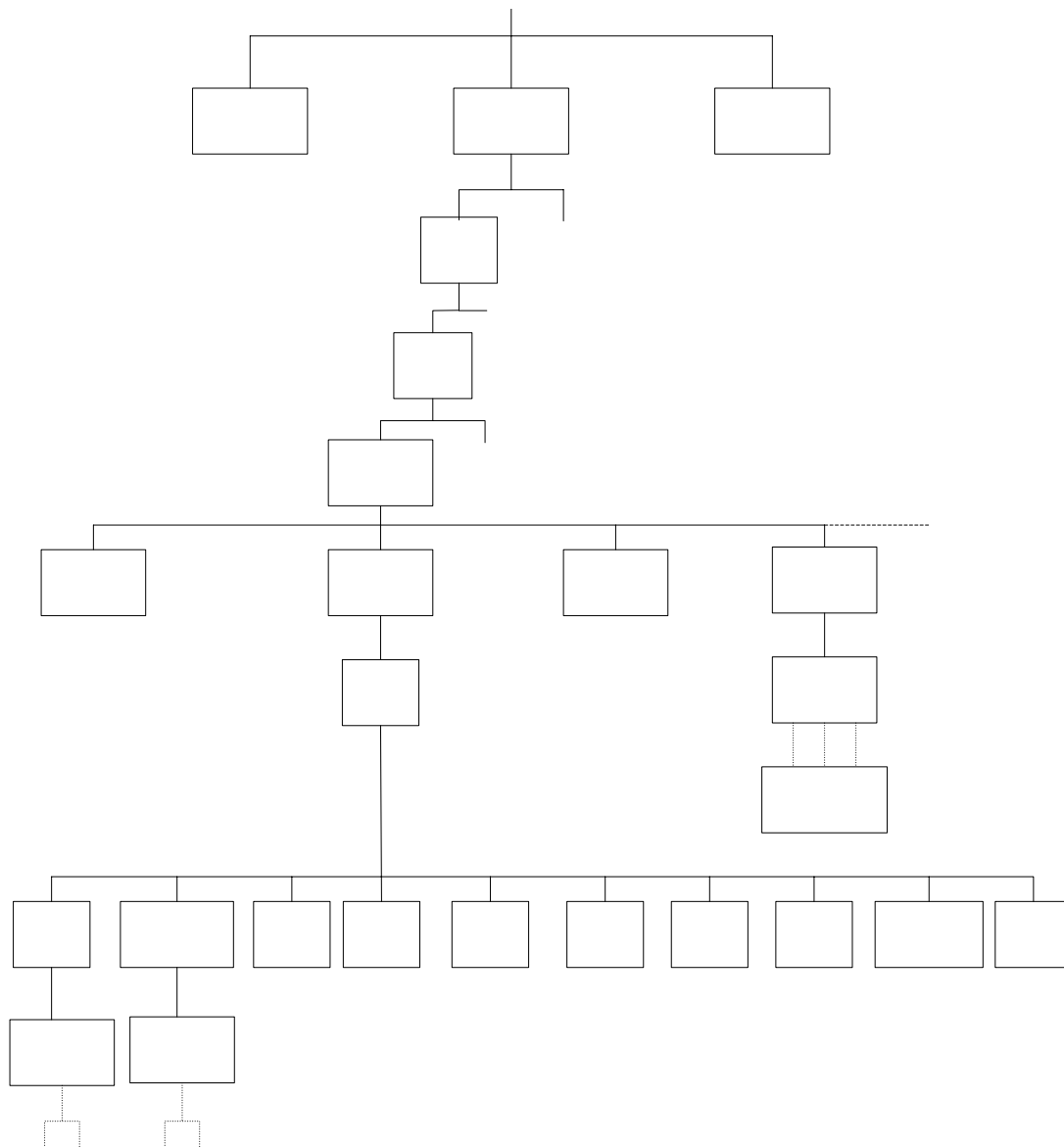
ภายใน MIB ประกอบด้วยตัวแปรจำนวนมากที่เรียกโดยทั่วไปว่า อ็อบเจกต์จัดการ (managed object) อ็อบเจกต์ในความหมายนี้เป็นชื่อที่ใช้เรียกตัวแปรและลักษณะเฉพาะของตัวแปร ใน MIB โดยไม่เกี่ยวข้องกับเชิงวัตถุพิสัย (object oriented) แต่อย่างใด โดยจะพิจารณาอ็อบเจกต์ใน SNMP มีลักษณะเช่นเดียวกับเรคคอร์ดในฐานข้อมูล

แต่ละอ็อบเจกต์ จะมีชื่อเรียกเฉพาะเรียกว่า อ็อบเจกต์ไอดีไฟเดอร์ (Object Identifier) หรือเรียกโดยย่อว่า ไอดีไฟเดอร์ (Identifier) เพื่อใช้อ้างอิงถึงอ็อบเจกต์นั้น

อ็อบเจกต์ทุกตัวมีนิยามที่กำหนด ชื่อ แบบข้อมูล สิทธิการเข้าถึง คำอธิบายลักษณะ และค่าข้อมูลการนิยามอ็อบเจกต์มีกฎเกณฑ์ตามข้อกำหนด โครงสร้างฐานข้อมูลสารสนเทศ ( Structure of Management Information: SMI ) [RFC 1155]

#### โครงสร้าง MIB

ข้อมูลประจำอุปกรณ์เครือข่ายชิ้นหนึ่งๆมีได้อย่างหลากหลาย อีกทั้งอุปกรณ์ต่างประเภทกันย่อมมีข้อมูลประจำอุปกรณ์แตกต่างกัน ดังนั้นการสอบถาม (อ่าน) หรือเปลี่ยนค่า (เขียน) ฐานข้อมูลจึงต้องมีรูปแบบมาตรฐานให้กับอุปกรณ์ทุกประเภท โครงสร้างต้นไม้แบบลำดับชั้นเป็นโครงสร้างที่เหมาะสมสำหรับใช้เป็นฐานข้อมูลเพื่อจัดเก็บตัวแปรเหล่านี้



รูปที่ 2.4 อ็อบเจกต์ไอดีไฟเบอร์ในโครงสร้างฐานข้อมูลสารสนเทศการจัดการ

ดังรูป 2.4 แสดงข้อมูลหรืออ็อบเจกต์ของเอสเอ็นเอ็มพีในโครงสร้างต้นไม้ซึ่งนิยมเรียกว่า มิบทรี(MIB Tree) แต่ละโหนดซึ่งแทนอ็อบเจกต์หนึ่งๆมีชื่อพร้อมทั้งตัวเลขฐานสิบกำกับประจำ โหนดเพื่อใช้อ้างอิง ยกเว้นรากซึ่งไม่มีชื่อกำกับ

ลำดับชั้นแรกจะมีโหนดหลัก 3 โหนดซึ่งกำหนดกลุ่มองค์กร 3 กลุ่มคือ ITU-T(0),ISO(1) และJoint-ISO-ITU-T(2) ภายใต้โหนด ISO มีโหนดลำดับที่ 3 คือ org(3) กำหนดองค์กรนานาชาติ และส่วนหนึ่งขององค์กรนี้คือ dod (6) หรือ Department of Defense และมี โหนด internet(1) เพื่อ กำหนดกลุ่มการจัดการเครือข่ายใน internet

เมื่อต้องการอ้างอิงถึงโหนดใดในโครงสร้าง ให้เขียนหมายเลขจากรากไปตามเส้นทางถึงโหนดนั้นและคั่นด้วยจุด ลำดับตัวเลขนี้เรียกว่า อ็อบเจกต์ไอดีไฟเอร์(Object Identifier) หรือ โอไอดี (OID)

ตัวอย่างเช่น 1.3.6.1.2.1.1 เป็นอ็อบเจกต์ไอดีไฟเอร์โดยมีชื่อที่สมนัยกันคือ iso.org.dod.internet.mgmt.mib-2.system โหนดที่อยู่ภายใต้ 1.3.6.1.2.1 หรือในกลุ่ม mib-2 เป็นโหนดสำหรับใช้งานเอสเอ็นเอ็มพี แต่ละโหนดจะมีโหนดย่อยเพื่ออ้างอิงถึงตัวแปร เช่น 1.3.6.1.2.1.1.1 คือตัวแปรsysDescr (System Description) ซึ่งเก็บคำอธิบายเกี่ยวกับอุปกรณ์นั้น

### กลุ่มในมิบ

มิบภายใต้ internet มีกลุ่มย่อยทั้งหมด 6 กลุ่มคือ

1. directory(1) สงวนไว้สำหรับใช้งานในอนาคต
2. mgmt(2) กลุ่มมิบที่ใช้ในการจัดการภายใต้เอสเอ็นเอ็มพีรุ่น 1
3. experimental(3) ใช้สำหรับการทดลอง
4. private(4) สำหรับผู้ผลิตกำหนดตัวแปรเฉพาะอุปกรณ์
5. security(5) ใช้ในระบบรักษาความปลอดภัย
6. SNMPv2(6) ใช้ในเอสเอ็นเอ็มพีรุ่น 2

ภายใต้กลุ่ม mib-2(1.3.6.1.2) บรรจุกลุ่มย่อยที่ใช้ในเอสเอ็นเอ็มพีซึ่งประกอบด้วย interface, at, ip และอื่นๆ

ความหมายของแต่ละกลุ่มอธิบายไว้ในตารางที่ 2-2 แต่ละกลุ่มประกอบด้วยตัวแปรซึ่งมีแบบต่างๆกันไป

| ลำดับ | ชื่อ         | ความหมาย                                      |
|-------|--------------|-----------------------------------------------|
| 1     | system       | ข้อมูลระบบ                                    |
| 2     | interface    | ข้อมูลอินเทอร์เฟซที่ใช้เชื่อมต่อ              |
| 3     | at           | ข้อมูลการแปลงแอดเดรส                          |
| 4     | ip           | ข้อมูลไอพี                                    |
| 5     | icmp         | ข้อมูลไอซีเอ็มพี                              |
| 6     | tcp          | ข้อมูลทีซีพี                                  |
| 7     | udp          | ข้อมูลยูดีพี                                  |
| 8     | egp          | ข้อมูลโปรโตคอลเกตเวย์ภายนอก                   |
| 10    | transmission | ข้อมูลสายสื่อสาร                              |
| 11    | SNMP         | ข้อมูลเอสเอ็นเอ็มพี                           |
| 16    | RMON         | ข้อมูลสำหรับใช้ตรวจสอบค่าประจำอุปกรณ์แบบรีโมต |

ตารางที่ 2.2 กลุ่มย่อยภายใต้ mgmt

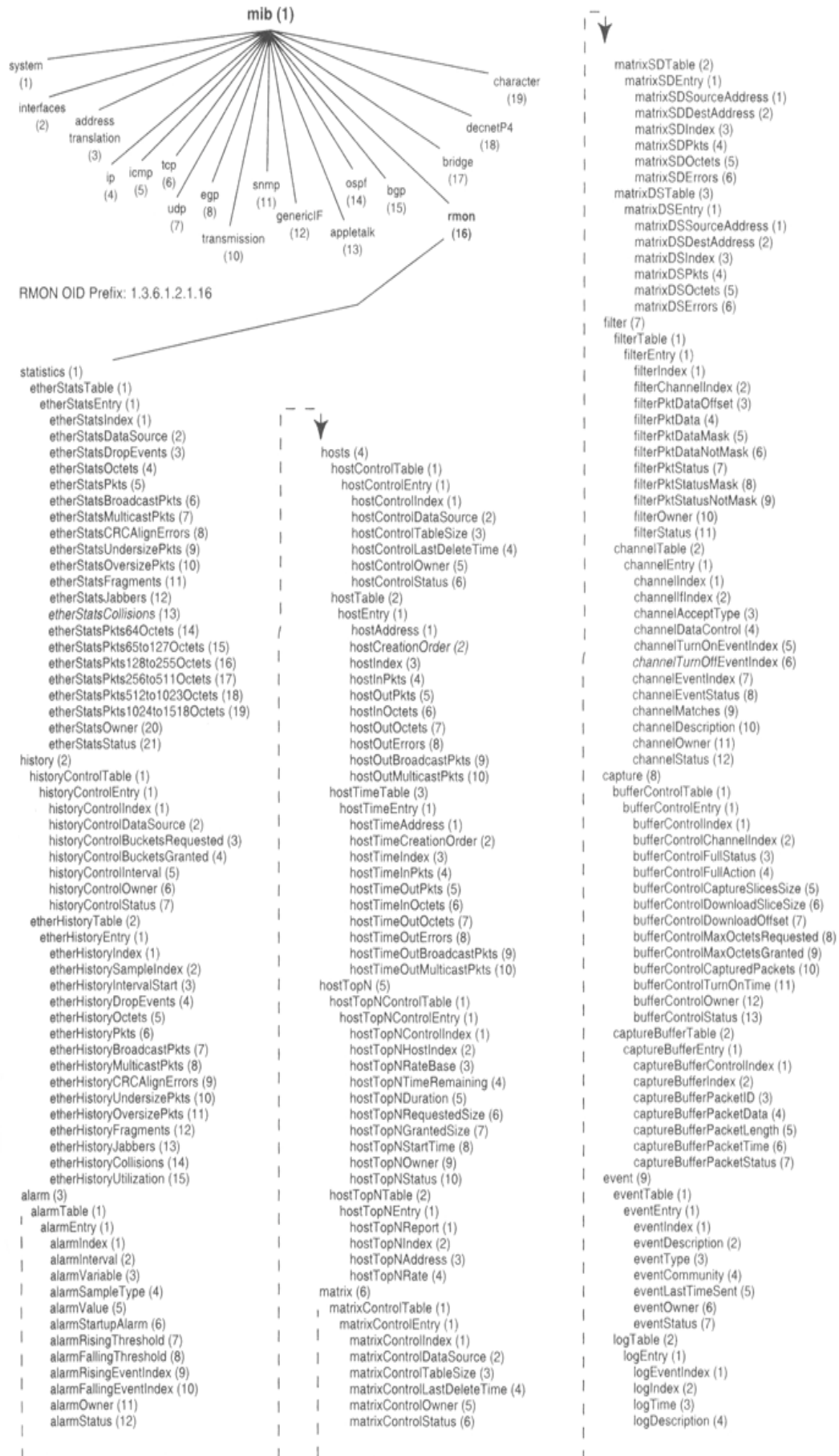
## RMON

Remote network monitoring (RMON) เป็นมิมที่สำคัญชุดหนึ่ง ซึ่งรวมอยู่ในมาตรฐานของเอสเอ็นเอ็มพี ใช้ในการตรวจสอบค่าประจำอุปกรณ์แบบรีโมต และทำการบันทึกค่าสถิติของอุปกรณ์บนเครือข่ายได้

ภายใต้กลุ่ม rmon (1.3.6.1.2.1.16) บรรจุกลุ่มย่อยที่ใช้ในRMONซึ่งประกอบด้วย 9 กลุ่มหลักด้วยกัน ดังตารางที่ 2.3 และรูปที่ 2.5

| ลำดับ | ชื่อ       | ความหมาย                                                                                                         |
|-------|------------|------------------------------------------------------------------------------------------------------------------|
| 1     | statistics | ข้อมูลทางสถิติ เช่น จำนวน และขนาดต่างๆของแพ็กเก็ตที่ได้รับ ทั้ง broadcasts , collisions , fragments ฯลฯ          |
| 2     | history    | บันทึกสถิติเป็นช่วงๆเพื่อนำมาวิเคราะห์                                                                           |
| 3     | alarm      | เปรียบเทียบสถิติกับค่าที่ตั้งไว้ซึ่งหากตรวจพบว่าสถิติมีค่าเกินกว่าที่ตั้งไว้จะทำการส่งสัญญาณเตือน(Alarm)         |
| 4     | hosts      | บันทึกสถิติเป็นรายชื่อเครื่องที่ใช้บริการเน็ตเวิร์ก รวมทั้ง MAC Address ของแต่ละเครื่องด้วย                      |
| 5     | hosTopN    | แสดงรายงานของเครื่องในเน็ตเวิร์กที่มีค่าทางสถิติสูงที่สุด โดยสามารถเลือกได้ว่าจะใช้ค่าใดเป็นเกณฑ์ในการแสดงรายงาน |
| 6     | matrix     | บันทึกสถิติของการสื่อสารระหว่างคู่ของเครื่องภายในระบบ                                                            |
| 7     | filter     | ยอมรับให้แพ็กเก็ตที่ตรงกับเงื่อนไขที่ได้ตั้งไว้ถูกตรวจสอบ                                                        |
| 8     | capture    | ทำการตรวจจับและส่งแพ็กเก็ตไปยัง Management console เมื่อแพ็กเก็ตตรงกับเงื่อนไขที่ได้ตั้งไว้                      |
| 9     | event      | เก็บสถิติของ Event ที่สร้างโดย RMON Probe                                                                        |

ตารางที่ 2.3 กลุ่มย่อยภายใต้ rmon



รูปที่ 2.5 โครงสร้างของกลุ่มย่อยภายใต้ rmon

## ชนิดของตัวแปรมิบ

แต่ละตัวแปรในเอสเอ็นเอ็มพีมีข้อมูลประจำ แบบข้อมูลที่ใช้อยู่ในเอสเอ็นเอ็มพีมีดังนี้

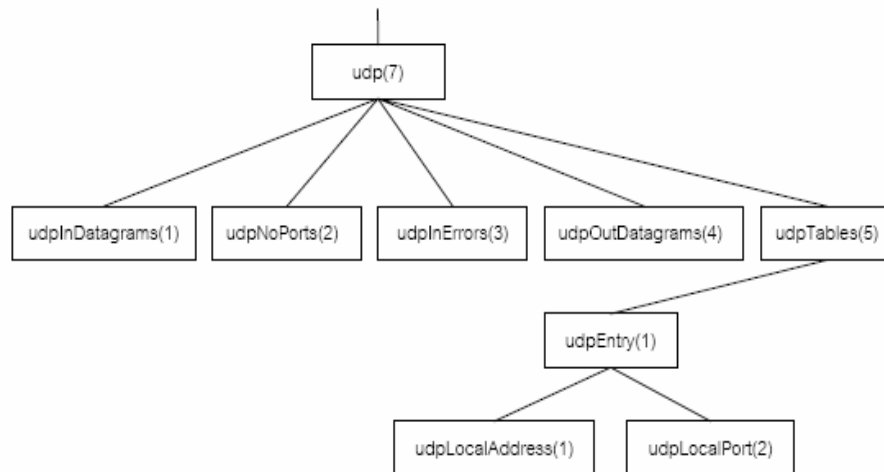
- Integer: จำนวนเต็มเช่นหมายเลขพอร์ตของโปรโตคอลทีซีพีหรือยูดีพี มีค่าได้ตั้งแต่ 0 ถึง 65535
- OctetString: สายอักขระขนาดตั้งแต่ 0 อ็อกเทต แต่ละอ็อกเทตมีค่าตั้งแต่ 0 ถึง 255 ตัวอย่างแบบข้อมูลสายอักขระได้แก่ รหัสผ่าน
- DisplayString: สายอักขระตั้งแต่ 0 อ็อกเทตแต่ละอ็อกเทตต้องเป็นรหัสแอสกี เอ็นวีที ข้อมูลประเภทนี้มีความยาวตั้งแต่ 0 ถึง 255 ตัวอักษร
- Null: ใช้บอกว่าตัวแปรนั้นไม่มีค่าข้อมูลใดอยู่ เช่นเมื่อสอบถามข้อมูลด้วยคำสั่ง Get หรือ GetNextRequest จะกำหนดแบบข้อมูลตัวแปรเท่ากับ null
- ObjectIdentifier: ชื่อตัวแปรในรูปแบบของการอ้างอิงแบบตัวเลขตามโครงสร้างมิบ
- IPAddress: สายอักขระ 4 อ็อกเทต แต่ละอ็อกเทตแทนไอพีแอดเดรสแต่ละตำแหน่ง
- PhysicalAddress: สายอักขระกำหนดฮาร์ดแวร์แอดเดรสเช่น อีเทอร์เน็ตแอดเดรส ใช้สายอักขระ 6 อ็อกเทต
- Counter: เลขจำนวนเต็มไม่คิดเครื่องหมาย มีค่าตั้งแต่ 0 ถึง  $2^{31} - 1$  (4,294,967,295) การใช้ข้อมูล counter เป็นแบบเพิ่มค่าขึ้นอย่างเดียว เมื่อเพิ่มถึงค่ามากที่สุดจะกลับเป็น 0 ใหม่
- Gauge: เลขจำนวนเต็มไม่คิดเครื่องหมาย มีค่าตั้งแต่ 0 ถึง  $2^{31} - 1$  โดยสามารถเพิ่มหรือลดค่าได้ แต่เมื่อเพิ่มไปสูงสุดแล้วจะคงค่าไว้จนกว่าจะถูกปรับค่ากลับมาเป็น 0 อีกครั้ง ตัวอย่างตัวแปรที่ใช้ค่านี้เช่น จำนวนการเชื่อมโยงทีซีพีที่อนุญาตให้มีได้
- TimeTicks: เลขจำนวนเต็มใช้นับเวลาให้หน่วยเศษหนึ่งส่วนร้อยของวินาที เช่น เวลานั้นตั้งที่ระบบเริ่มทำงาน
- Sequence: โครงสร้างแบบเร็คคอร์ด หรือคล้ายกับแบบข้อมูล struct ในภาษาซี
- Sequence of: โครงสร้างแบบตารางหรือมองในรูปของอาร์เรย์ เช่น ตารางเลือกเส้นทางของไอพี

## ตัวอย่างแบบข้อมูลอาร์เรย์

แบบข้อมูล sequence of ใช้กำหนดข้อมูลแบบเวกเตอร์ซึ่งสมาชิกทั้งหมดภายในมีข้อมูลแบบเดียวกัน หากสมาชิกมีแบบข้อมูลเบื้องต้น เช่น แบบจำนวนเต็มก็จะได้เวกเตอร์แบบมิติเดียว หรือหากสมาชิกมีข้อมูลแบบ sequence ก็จะได้อาร์เรย์ 2 มิติ (ตาราง)



ตัวอย่างของตัวแปรโครงสร้างอาร์เรย์ในมิมมียู่หลายตัวแปร แต่จะยกตัวอย่างเฉพาะตัวแปรที่มีขนาดเล็กเพื่อที่จะทำความเข้าใจได้ง่ายได้แก่ udpTable ซึ่งสังกัดอยู่ภายใต้กลุ่ม udp ตามโครงสร้างข้อมูล ดังรูปที่ 2.5



รูปที่ 2.6 กลุ่ม udp

udpTable มีแบบข้อมูล sequence of และภายใต้ udpTable มี udpEntry ซึ่งมีแบบข้อมูล sequence โดยประกอบด้วย udpLocalAddress และ udpLocalPort

udpLocalAddress มีแบบข้อมูล IPAddress ใช้กำหนดไอพีแอดเดรสที่รอให้บริการส่วนของ udpLocalPort กำหนดหมายเลขพอร์ตดังนั้น udpTable จึงมีโครงสร้างเป็นอาร์เรย์ 2 มิติหรือเขียนด้วย ตารางดังรูปที่2.6

|          | udpLocalAddress | udpLocalPort |
|----------|-----------------|--------------|
| udpEntry | (IpAddress)     | (Integer)    |
| udpEntry | ...             | ...          |
| udpEntry | ...             | ...          |
| udpEntry | ...             | ...          |
| udpEntry | ...             | ...          |

รูปที่ 2.7 udpTable ในรูปอาร์เรย์ 2 มิติ (ตาราง)

### การอ้างอิงถึงค่าในอ็อบเจกต์ (Instance Identification)

ในการอ้างอิงถึงค่าของอ็อบเจกต์ในฐานข้อมูลสารสนเทศการจัดการของเอสเอ็นเอ็มพีนั้นจะอาศัย การอ้างอิงของอินสแตนซ์ไอดีไฟเดอร์ (Instance Identifier)

โดยการอ้างอิงถึงค่าของอ็อบเจกต์แบบปกติ (Simple Object Value) โดยทั่วไปจะใช้อินสแตนซ์ไอดีไฟเดอร์ที่ประกอบไปด้วยค่าของ อ็อบเจกต์ไอดีไฟเดอร์ (Object Identifier) แล้

วปิดท้ายด้วย 0 เช่น การอ้างถึงค่าในอ็อบเจ็กต์ sysDescr ด้วยค่าอินสแตนท์ไอดีเอ็นดีไฟเออร์คือ 1.3.6.1.2.1.1.1.0 ซึ่งก็ คือ จะประกอบด้วยค่าอ็อบเจ็กต์ไอดีเอ็นดีไฟเออร์ของอ็อบเจ็กต์ sysDescr คือ 1.3.6.1.2.1.1.1 แล้วต่อท้ายด้วย 0 นอกจากนั้นยังอาจเขียนในรูปของชื่อได้ คือ iso.org.dod.internet.mgmt.mib-2.system.sysDescr.0 หรือเขียนสั้นๆเพียง sysDescr.0 ก็ได้

สำหรับการอ้างถึงค่าของอ็อบเจ็กต์ในรูปแบบตาราง (Sequence of Object Value) นั้น เอสเอ็นเอ็มพีไม่อนุญาตให้อ้างอิงทั้งตารางได้ในคราวเดียว (เช่น ไม่สามารถอ้างเพียง udp เพื่อขอข้อมูลทั้งอาร์เรย์) การอ้างอิงจะต้องทำที่อ็อบเจ็กต์ที่เป็นโหนดปลายเท่านั้น (Leaf object) หรือต้องเจาะจงถึงค่าในตารางนั้นเลย เช่น udpLocalAddress หรือ udpLocalPort และอ้างไปที่ละค่า

ตัวอย่างเช่น เอเจนต์พร้อมที่จะรับยูดีพีเดทาแกรมทุกอินเทอร์เฟซสำหรับบริการ BOOTP (67), TFTP (69) และ SNMP (151) เอเจนต์จะจัดเก็บค่านี้อยู่ในมิบที่โหนด udpTable จำนวน 3 ค่า ดังตารางที่ 2.4

| udpLocalAddress | udpLocalPort |
|-----------------|--------------|
| 0.0.0.0         | 67           |
| 0.0.0.0         | 69           |
| 0.0.0.0         | 151          |

ตารางที่ 2.4 ตัวอย่างค่าใน udpTable

หาก สถานีจัดการ ต้องการถามว่าเอเจนต์พร้อมรับยูดีพีเดทาแกรมสำหรับบริการใดบ้างก็ให้ใช้คำสั่ง GetRequest หรือ GetNextRequest โดยระบุไอดีเอ็นดีไฟเออร์ที่อ้างถึง udpLocalAddress และ udpLocalPort ดังตารางที่ 2.5

| อ็อบเจ็กต์ไอดีเอ็นดีไฟเออร์     | อ็อบเจ็กต์ไอดีเอ็นดีไฟเออร์ แบบย่อ | ค่า     |
|---------------------------------|------------------------------------|---------|
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.67  | udpLocalAddress.0.0.0.0.67         | 0.0.0.0 |
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.69  | udpLocalAddress.0.0.0.0.69         | 0.0.0.0 |
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.161 | udpLocalAddress.0.0.0.0.161        | 0.0.0.0 |
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.67  | udpLocalPort.0.0.0.0.67            | 67      |
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.69  | udpLocalPort.0.0.0.0.69            | 69      |
| 1.3.6.1.2.1.7.5.1.1.0.0.0.0.161 | udpLocalPort.0.0.0.0.161           | 161     |

ตารางที่ 2.5 อ็อบเจ็กต์ไอดีเอ็นดีไฟเออร์อ้างอิงค่าใน udpTable

จะสังเกตว่าการอ้างอิงค่าด้วยอ็อบเจ็กต์ไอดีเอ็นดีไฟเออร์ไม่ได้ใช้ดัชนีชี้ตำแหน่งเหมือนการอ้างอาร์เรย์ในภาษาคอมพิวเตอร์ หากแต่ใช้ “ค่าข้อมูล” ที่อยู่ในตารางมาเป็นตัวกำหนดดัชนี (Index)

ในขั้นตอนแรกของการอ้างอิงค่าจะใช้ไอดีเอ็นดีไฟเบอร์ udpTable หาจุดเริ่มต้นของตาราง ก่อนด้วยคำสั่ง GetNextRequest ( udpTable) เพื่อให้ได้ข้อมูลค่าแรก จากนั้นจึงนำค่าที่ได้มาใช้เป็นไอดีเอ็นดีไฟเบอร์สำหรับค่าถัดไปด้วยคำสั่ง GetNextRequest ( udpLocalAddress.0.0.0.0.67 ) และทำซ้ำเช่นนี้จนกระทั่งได้ค่าสุดท้ายในตาราง จุดสิ้นสุดของตารางก็ตรวจได้จากไอดีเอ็นดีไฟเบอร์ที่เป็นชื่อใหม่

คำสั่ง GetNextRequest จะนำค่าถัดไปมาโดยไม่ต้องเจาะจงค่าโดยอำนวยความสะดวกอย่างมากต่อตัวแปรที่เป็นชนิด sequence และ sequence of แต่ลำดับการนำค่าของ GetNextRequest จะมีลำดับเริ่มที่คอลัมน์แรกจนสิ้นสุดทุกแถวก่อนที่จะขึ้นคอลัมน์ถัดไป ดังนั้นการใช้จาก GetNextRequest ในกรณีของค่าในรูปที่ จะมีลำดับของค่าที่ได้ดังตารางที่ 2.6

| ลำดับคำสั่งสอบถามด้วย GetNextRequest | ค่าที่ได้                           |
|--------------------------------------|-------------------------------------|
| udpTable                             | udpLocalAddress.0.0.0.0.67=0.0.0.0  |
| udpLocalAddress.0.0.0.0.67           | udpLocalAddress.0.0.0.0.69=0.0.0.0  |
| udpLocalAddress.0.0.0.0.69           | udpLocalAddress.0.0.0.0.161=0.0.0.0 |
| udpLocalAddress.0.0.0.0.161          | udpLocalPort.0.0.0.0.67=67          |
| udpLocalPort.0.0.0.0.67              | udpLocalPort.0.0.0.0.69=69          |
| udpLocalPort.0.0.0.0.69              | udpLocalPort.0.0.0.0.161=161        |
| udpLocalPort.0.0.0.0.161             | egpInMsgs.0=161                     |

ตารางที่ 2.6 การสอบถามค่าจากตารางด้วยGetNextRequest ตามลำดับคอลัมน์ก่อน

## 2.2.4 การแทนข้อมูลด้วย ASN.1

ไอเอสโอและซีซีไอทีที่กำหนดวิธีการนิยามชนิดของตัวแปรโดยใช้ไวยากรณ์ ASN.1 (Abstract Syntax Notation One) ซึ่งเป็นเป็นเสมือนภาษอธิบายแบบข้อมูลที่ไม่ขึ้นกับฮาร์ดแวร์ต้นกำเนิดของ ASN.1 นำมาใช้นิยามชุดโปรโตคอลของไอเอสโอ แต่สำหรับเอสเอ็นเอ็มพีจำใช้เพียงไวยากรณ์เพียงบางส่วนของ ASN.1 เท่านั้น

การใช้ ASN.1 ช่วยให้ผู้นำมามาตรฐานไปใช้เข้าใจถึงสิ่งที่ผู้สร้างมาตรฐานกำหนดไว้โดยไม่เกิด ความกำกวมในเรื่องของความหมายและการแทนข้อมูล เช่นการกำหนดตัวแปรชนิด integer จะต้องกำหนดให้แน่นอนว่ามีค่าอยู่ในช่วงใด เพราะว่าคอมพิวเตอร์ต่างชนิดกันอาจมีความแตกต่างกันในการแทนข้อมูลและช่วงของตัวแปรที่แตกต่างกัน มีบ๊อบเจ็กในเอสเอ็นเอ็มพีมีรูปแบบที่กำหนดด้วย ASN.1ตัวอย่างต่อไปนี้เป็นนิยามของอ็อบเจ็ก sysContact

sysContact OBJECT-TYPE

SYNTAX DisplayString (SIZE (0..255))

ACCESS read-write

STATUS mandatory

DESCRIPTION

"The textual identification of the contact person  
for this managed node, together with information  
on how to contact this person."

::= { system 4 }

นิยามข้างต้นมีความหมายดังนี้

- SYNTAX : กำหนดแบบข้อมูลของตัวแปรเช่นจำนวนเต็มหรืออักขระ ในที่นี้คือ DisplayString
- ACCESS : แบบการเข้าใช้ซึ่งอาจเป็น read-only(อ่านอย่างเดียว), read-write(อ่านเขียนได้), write-only(เขียนอย่างเดียว) หรือ not-accessible (ห้ามเข้าถึง) เป็นต้น
- STATUS : กำหนดสถานะของตัวแปรว่าจำเป็นต้องมีตัวแปรนี้หรือไม่ ค่าที่เป็นไปได้เช่น mandatory(จำเป็น), optional(ออฟชั่นซึ่งมีหรือไม่ก็ได้), depreiate (จำเป็นต้องมีแต่อาจยกเลิกในรุ่นถัดไป), obsoleted (ไม่จำเป็นเนื่องจากยกเลิกไม่ใช้แล้ว)
- DESCRIPTION : ข้อความอธิบายตัวแปร
- บรรทัดสุดท้ายของนิยามกำหนดว่าตัวแปร sysContact จะเชื่อมกับโครงสร้างต้นไม้ต่อจากโหนด system และมีค่าเท่ากับ 4 ซึ่งแสดงถึงไอเด็นติไฟเออร์ประจำ sysContact คือ iso.org.dod.internet.mgmt.mib-2.system.4 หรือ 1.3.6.1.2.1.1.4

อีกตัวอย่างหนึ่งต่อไปนี้แสดง โครงสร้างมิบบางส่วนของกลุ่มภายใต้โหนด mib-2 โปรดสังเกตว่าสัญลักษณ์ “--” ที่ปรากฏในนิยามใช้เป็นส่วนหมายเหตุแสดงคำอธิบาย

-- groups in MIB-II

```
system    OBJECT IDENTIFIER ::= { mib-2 1 }  
interfaces OBJECT IDENTIFIER ::= { mib-2 2 }  
at        OBJECT IDENTIFIER ::= { mib-2 3 }  
ip        OBJECT IDENTIFIER ::= { mib-2 4 }  
icmp      OBJECT IDENTIFIER ::= { mib-2 5 }  
tcp       OBJECT IDENTIFIER ::= { mib-2 6 }  
udp       OBJECT IDENTIFIER ::= { mib-2 7 }  
egp       OBJECT IDENTIFIER ::= { mib-2 8 }
```

```

-- historical (some say hysterical)

-- cmot    OBJECT IDENTIFIER ::= { mib-2 9 }

transmission OBJECT IDENTIFIER ::= { mib-2 10 }

snmp      OBJECT IDENTIFIER ::= { mib-2 11 }

-- the System group

-- Implementation of the System group is mandatory for all

-- systems. If an agent is not configured to have a value

-- for any of these variables, a string of length 0 is

-- returned.

sysDescr OBJECT-TYPE

    SYNTAX  DisplayString (SIZE (0..255))

    ACCESS  read-only

    STATUS  mandatory

DESCRIPTION

    "A textual description of the entity. This value

    should include the full name and version

    identification of the system's hardware type,

    software operating-system, and networking

    software. It is mandatory that this only contain

    printable ASCII characters."

    ::= { system 1 }

```

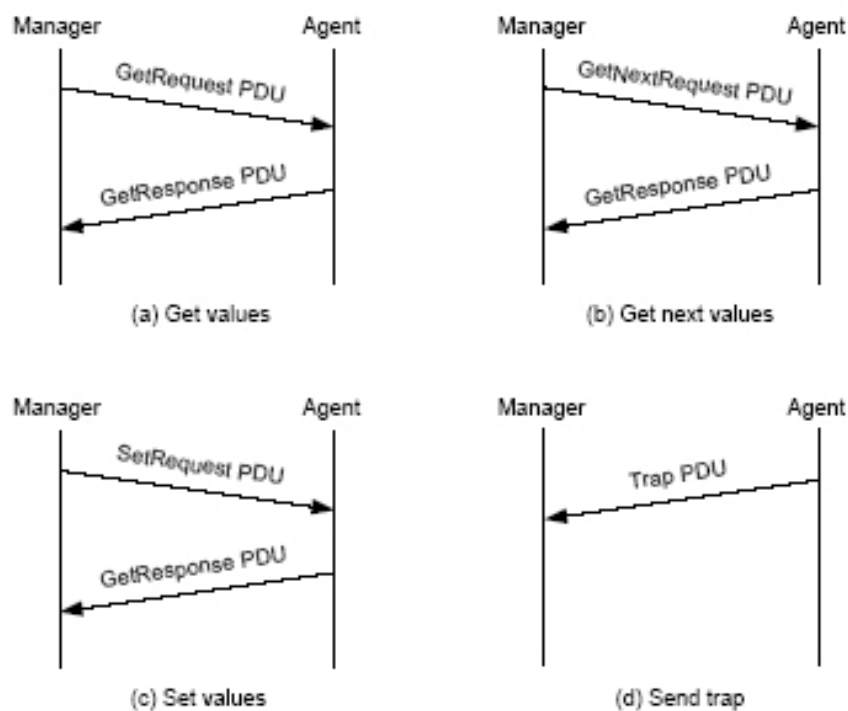
## 2.2.5 ลักษณะของโปรโตคอล (Protocol Specification)

การติดต่อระหว่างสถานีจัดการกับเอเจนต์มีรูปแบบในการติดต่อที่เรียกว่า protocol data units หรือ พีดียู (PDU) หลายรูปแบบด้วยกันตามวัตถุประสงค์ในการติดต่อ แบบของการติดต่อใน เอสเอ็นเอ็มพีรุ่น 1 มี 5 แบบคือ

1. GetRequest PDU ใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่าย
2. GetNextRequest PDU ใช้สอบถามข้อมูลที่เรียงเป็นลำดับ เช่น ข้อมูลที่เก็บอยู่ในรูปตาราง หรือในกรณีที่ไม่ทราบชื่อตัวแปรที่แน่ชัด
3. GetResponse PDU เอเจนต์ส่งคำตอบกลับมายังผู้สอบถาม
4. SetRequest PDU ใช้เปลี่ยนแปลงค่าของอ็อบเจกต์ที่เอเจนต์รับผิดชอบอยู่

5. Trap PDU ใช้แจ้งเหตุการณ์ที่เกิดขึ้นในระบบเครือข่าย เช่นการเริ่มต้นทำงานใหม่ของอุปกรณ์ หรือเส้นทางขัดข้อง

เอสเอ็นเอ็มพีอาศัยโปรโตคอลยูดีพี โดยเอเจนต์จะใช้พอร์ตหมายเลข 161 สำหรับการติดต่อในแบบที่ 1 ถึง 4 จากสถานีจัดการเครือข่ายและสถานีจัดการเครือข่ายจะใช้พอร์ตหมายเลข 162 สำหรับการติดต่อในแบบที่ 5 จากเอเจนต์ โดยจะมีลำดับการทำงานในการติดต่อทั้ง 5 รูปแบบ ดังรูปที่ 2.7



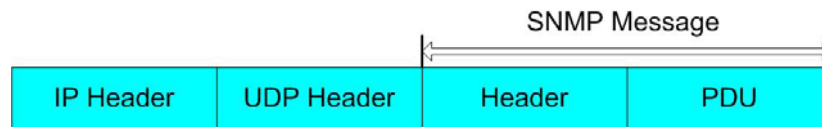
รูปที่ 2.8 ลำดับการทำงานของ SNMP PDU

ส่วนเอสเอ็นเอ็มพี รุ่น 2 ได้รับการปรับปรุงให้มีความสามารถเพิ่มขึ้นจากรุ่นแรก ฟังก์ชันที่เพิ่มขึ้นได้แก่ คำสั่ง *GetBulkRequest* เพื่อสอบถามค่าโดยกำหนดจำนวนอ็อบเจกต์ที่ต้องการได้ แทนที่จะต้องใช้ *GetNextRequest* ซ้ำหลายครั้ง นอกจากนี้ ยังมี *InformRequest* สำหรับการสื่อสารระหว่างสถานีจัดการเครือข่าย เพื่อช่วยในการบริหารแบบกระจายภาระงาน จุดเด่นอีกส่วนหนึ่งที่มีในเอสเอ็นเอ็มพี รุ่น 2 ได้แก่ การรักษาความปลอดภัยโดยการเข้ารหัสข้อความ และการพิสูจน์ตัวตน รวมทั้งการขยายกลุ่ม MIB เพิ่มเติม [ RFC 1907 ]

#### การเอ็นแคปซูล (Encapsulation)

การเอ็นแคปซูลคำสั่งและข้อมูลในเอสเอ็นเอ็มพีมีวิธีตามรูปที่ 2.8 พอร์มของเอสเอ็นเอ็มพี ประกอบด้วย 2 ส่วนคือ เฮดเดอร์และพืดยู เฮดเดอร์ประกอบด้วยฟิลด์ย่อยสองฟิลด์คือ

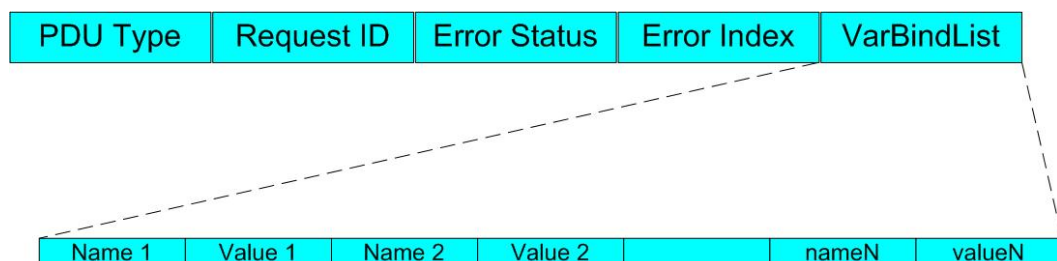
1. Version รุ่นของโพรโทคอลที่ใช้ ถ้าเป็นโพรโทคอลรุ่น 1 จะมีค่า 0 หากเป็นรุ่น 2 จะมีค่า 1
2. Community รหัสผ่านในรูปสายอักขระเพื่อให้เอเจนต์ใช้ในการพิสูจน์ตัวตนว่าข้อความที่ส่งมามีสิทธิ์ในการสอบถาม หรือเปลี่ยนแปลงข้อมูลหรือไม่ ซึ่งรายละเอียดได้อธิบายไว้ในหัวข้อ Communities และ Community Names ที่ผ่านมา



รูปที่ 2.9 การเอ็นแคปซูลเตเอสเอ็นเอ็มพี

ในส่วนของพีดียูประกอบด้วยฟิลด์ย่อยตามชนิดของข้อความ หากเป็นข้อความ GetRequest PDU, GetNextRequest PDU, GetResponse PDU และ SetRequest PDU จะมีโครงสร้างเดียวกัน รูปที่ 2.9 แสดงโครงสร้างของพีดียูโดยแต่ละฟิลด์มีความหมายดังนี้

- PDU type รูปแบบการติดต่อ (1 ถึง 5)
- Request ID กำหนดบอกหมายเลขข้อความเพื่อใช้จับคู่เมื่อรับคำตอบกลับมา
- Error Status สถานะความผิดพลาดที่เกิดขึ้นโดยรหัสความผิดพลาดและสถานะผิดพลาดที่ใช้ในเอสเอ็นเอ็มพีจะแสดงในตารางที่ 3 สำหรับข้อความ GetRequest PDU, GetNextRequest PDU และ SetRequest PDU จะมีค่าในฟิลด์นี้เป็น 0 เสมอ
- Error Index ดรรชนีชี้ค่าผิดพลาดที่เกิดขึ้นเกิดจากตัวแปรตัวลำดับที่เท่าไรของตัวแปรทั้งหมดที่สอบถามไปสำหรับข้อความ GetRequest PDU, GetNextRequest PDU และ SetRequest PDU จะมีค่าในฟิลด์นี้เป็น 0 เสมอ
- VarBindList ค่าผูกพันตัวแปร(variable binding) แสดงอยู่ในรูปของการอ้างอิงตัวแปรหรืออ็อบเจ็กต์ (name) และค่าของตัวแปร (value) ต่อเนื่องกันไปเป็นรายการสำหรับข้อความ GetRequest PDU, GetNextRequest PDU และ SetRequest PDU ค่าของตัวแปรจะมีค่าเป็น NULL เสมอ



รูปที่ 2.10 แสดงโครงสร้างของพีดียู GetRequest PDU, GetNextRequest PDU, GetResponse PDU และ SetRequest PDU

| รหัสผิดพลาด | ชื่อ       | คำอธิบาย                                                 |
|-------------|------------|----------------------------------------------------------|
| 0           | noError    | ไม่มีข้อผิดพลาด                                          |
| 1           | tooBig     | เอเจนต์ไม่สามารถส่งคำตอบได้ในเฟรมเดียว                   |
| 2           | noSuchName | ไม่มีตัวอ็อบเจ็กต์ที่ต้องการสอบถามอยู่ในฐานข้อมูล        |
| 3           | badValue   | ค่าที่กำหนดให้อ็อบเจ็กต์ไม่ถูกต้อง                       |
| 4           | readOnly   | เปลี่ยนค่าอ็อบเจ็กต์ไม่ได้เพราะอ่านค่าได้เพียงอย่างเดียว |
| 20          | genErr     | มีข้อผิดพลาดอื่นๆเกิดขึ้น                                |

ตารางที่ 2.7 รหัสและสถานะความผิดพลาดในเอสเอ็นเอ็มพี

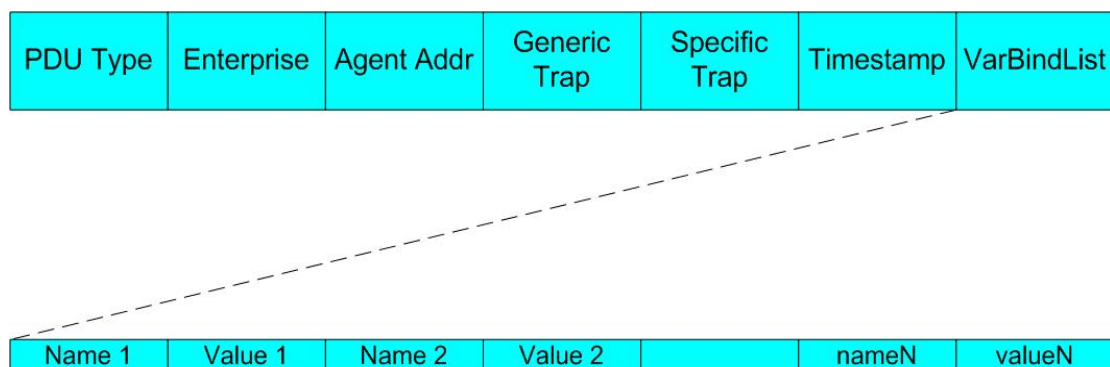
สำหรับข้อความ Trap PDU มีลักษณะแตกต่างกันออกไปดังรูปที่ 2.10 โดยแต่ละฟิลด์มีความหมายดังนี้

- Enterprise ชนิดของตัวแปรที่สร้าง Trap นี้ขึ้นมา โดยค่าในฟิลด์นี้จะอ้างอิงกับค่าในอ็อบเจ็กต์ sysObjectID
- Agent Addr ค่า IP Address ของอ็อบเจ็กต์ที่สร้าง Trap นี้ขึ้นมา
- Generic Trap ชนิดของ Trap ที่เกิดขึ้นโดยชนิดของ Trap และรหัสของ Trap ที่ใช้ในเอสเอ็นเอ็มพีจะแสดงในตารางที่ 2.8
- Specific Trap ชนิดของเหตุการณ์ผิดปกติเกิดขึ้นกับ enterprise-specific
- Time-stamp เวลาที่ใช้ไปทั้งหมดตั้งแต่การเริ่มการติดต่อในเครือข่ายรวมถึงเวลาที่ใช้ในการสร้าง Trap โดยค่าดังกล่าวจะเก็บอยู่ในอ็อบเจ็กต์ sysUpTime



| รหัส Trap | ชื่อ                  | คำอธิบาย                                                                                                                                                           |
|-----------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0         | coldStart             | มีการเริ่มต้นการทำงานใหม่ ด้วยสาเหตุของการเปลี่ยนแปลงค่าในเอเจนต์ หรือมีการเปลี่ยนแปลงโปรโตคอลเอ็นจิน เช่น การ restart อุปกรณ์หลังจากเกิดการล้มเหลวของระบบ (crash) |
| 1         | warmStart             | มีการเริ่มต้นการทำงานใหม่ แต่ไม่ได้เกิดจากสาเหตุของการเปลี่ยนแปลงค่าในเอเจนต์ หรือโปรโตคอลเอ็นจิน เช่น การ restart routine                                         |
| 2         | linkDown              | การเชื่อมต่อของเอเจนต์มีปัญหา จะมีการค่าของอ็อบเจ็กต์ ifIndex เพื่อบอกจุดเชื่อมต่อ (interface) ที่มีปัญหา                                                          |
| 3         | linkUp                | การเชื่อมต่อของเอเจนต์กลับมาใช้งานได้ จะมีการค่าของอ็อบเจ็กต์ ifIndex เพื่อบอกจุดเชื่อมต่อ (interface) ที่มีการกลับมาใช้งานได้                                     |
| 4         | authenticationFailure | การพิสูจน์ตัวตนของ message ที่เข้ามาไม่ผ่านหรือมีการผิดพลาดเกิดขึ้น                                                                                                |
| 5         | egpNeighborLoss       | โปรโตคอลเกตเวย์ภายนอก (External gateway protocol) มีปัญหา                                                                                                          |
| 6         | enterpriseSpecific    | มีเหตุการณ์ผิดปกติเกิดขึ้นกับ enterprise-specific โดยจะมีการประกาศชนิดของเหตุการณ์ผิดปกติเกิดขึ้นในฟิลด์ specific-trap                                             |

ตารางที่ 2.8 รหัส และชนิดของ Trap ในเอสเอ็นเอ็มพี



รูปที่ 2.11 โครงสร้างฟิลด์ของคำสั่ง Trap PDU

โปรดสังเกตว่าในที่นี้ไม่ได้กล่าวขนาดความยาวของแต่ละฟิลด์ เพราะทุกฟิลด์ในเอสเอ็นเอ็มพีต้องเข้ารหัสและจะได้ขนาดของแต่ละฟิลด์ที่มีความยาวแตกต่างกันไปตามชนิดข้อมูล

### **กลไกในการส่ง SNMP Message (Transmission of an SNMP Message)**

กลไกในการส่ง SNMP Message ของ PDU ทั้ง 5 แบบในส่วนของโปรโตคอลเอ็นจิน โดยการทำงานจะมีขั้นตอนดังต่อไปนี้

1. สร้าง PDU ตามโครงสร้างที่กำหนดไว้ใน ASN.1 (Abstract Syntax Notation One)
2. PDU ที่สร้างในขั้นตอนที่หนึ่งรวมค่า transport addresses ของต้นทางและปลายทาง และ Community name จะถูกส่งไปให้ส่วนของการบริการพิสูจน์ตัวตน (Authentication service) ซึ่งจะทำการปรับเปลี่ยนข้อมูลที่จำเป็นในการแลกเปลี่ยน เช่น การเข้ารหัสลับ (encryption) หรือการสร้างไคด์ที่ใช้ในการพิสูจน์ตัวตน หลังจากการปรับเปลี่ยนเสร็จก็จะคืนค่ากลับมายังโปรโตคอลเอ็นจิน
3. โปรโตคอลเอ็นจินจะสร้าง message ของเอสเอ็นเอ็มพีโดยการรวมฟิลด์รุ่นของโปรโตคอล (Version), Community name, และผลลัพธ์ที่ได้ในขั้นตอนที่ 2
4. ทำการเข้ารหัส message เอสเอ็นเอ็มพีที่ได้จากขั้นตอนที่ 3 โดยวิธีการ Basic Encoding Rule (BER) แล้วส่ง message ที่เข้ารหัสแล้วไปยังโปรโตคอลในชั้น Transport ต่อไป

กลไกในการรับ SNMP Message ของ PDU ในส่วนของโปรโตคอลเอ็นจิน โดยการทำงานจะมีขั้นตอนดังต่อไปนี้

1. ตรวจสอบความถูกต้องทางไวยากรณ์ขั้นพื้นฐานของ message โดยจะกำจัด message ที่ถ้าพบความผิดพลาด
2. รุ่นของโปรโตคอล (Version) ที่ใช้ โดยจะกำจัด message ที่ถ้าพบว่าเป็นคนละรุ่นกัน
3. โปรโตคอลเอ็นจินจะนำค่าของ use name, ส่วนของ PDU และ transport addresses ของต้นทางและปลายทาง ส่งไปให้ส่วนของการบริการพิสูจน์ตัวตน (Authentication service)
- ถ้าการพิสูจน์ตัวตนล้มเหลว ส่วนของการบริการพิสูจน์ตัวตนจะส่งสัญญาณบอกไปยังโปรโตคอลเอ็นจินเพื่อให้ทำการสร้าง Trap และจะกำจัด message นั้นทิ้ง
- ถ้าการพิสูจน์ตัวตนสำเร็จ ส่วนของการบริการพิสูจน์ตัวตนจะคืนค่า PDU ที่อยู่ในโครงสร้างของ ASN.1 มาให้กับโปรโตคอลเอ็นจิน
4. ตรวจสอบความถูกต้องทางไวยากรณ์ขั้นพื้นฐานของ PDU โดยจะกำจัด PDU ที่ถ้าพบความผิดพลาด หลังจากนั้นจะนำชื่อของ Community name เพื่อจัดสรร

รูปแบบนโยบายการเข้าถึง (Access policy) ตาม Community ของ PDU นั้น และ  
ทำการประมวลผลตามคำสั่งใน PDU ต่อไป

### 2.2.6 การเข้ารหัสโดยใช้ BER (Basic Encoding Rules)

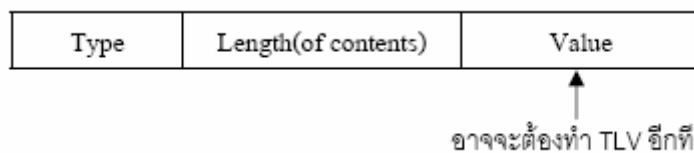
การส่งข้อมูลใน SNMP ไม่ได้ส่งในรูปแบบเป็น ออกเขตของของตัวเลขโดยตรง หากแต่ฝ่ายส่งต้องเข้ารหัสข้อมูลเพื่อนำส่งข้อมูลออก และ ถอดรหัสที่ฝ่ายรับ

#### โครงสร้างการเข้ารหัส

การเข้ารหัสตามแบบ BER จะมีข่าวสารกำกับอยู่ในตัวว่าเป็นข้อมูลชนิดใด และ มีความยาวเท่าใด ฟิลด์ที่ผ่านการเข้ารหัสแล้วประกอบด้วยค่า 3 ส่วน ดังรูปที่ 2.11 คือ

- ชนิดของข้อมูล (type หรือ tag หรือ identifier)
- ความยาวของข้อมูล (length)
- ตัวข้อมูล (value หรือ contents)

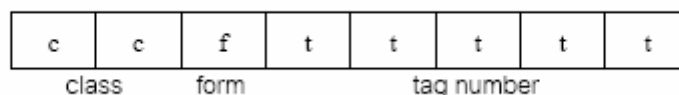
โครงสร้างรหัสเหล่านี้ เรียกว่า โครงสร้าง TLV (TLV : type-length-value structure) ค่าในส่วนของฟิลด์ value อาจจะต้องผ่านการเข้ารหัสค่าตามโครงสร้าง TLV ด้วยเช่นกัน



รูปที่ 2.12 โครงสร้างของ TLV

#### ฟิลด์กำหนดชนิดข้อมูล (Type)

การกำหนดค่าให้กับฟิลด์ Type ขนาด 8 bit มีการจัดวางตำแหน่งบิต เพื่อให้ได้ค่าตัวเลขที่ใช้แทนกลุ่มชนิดข้อมูล แบ่ง เป็น 3 ส่วนย่อย ดังรูปที่ 2.12



รูปที่ 2.13 รูปแบบการเข้ารหัสประเภทชนิดข้อมูล

1. ฟิลด์ class 2 bits แรก : กำหนดประเภทข้อมูล มี 4 แบบ คือ
  - 00 = ประเภท Universal เช่น ตัวเลขทั่วไป
  - 01 = ประเภท Application ข้อมูลสำหรับใช้กับ application program

- 10 = ประเภท Context Specific ข้อมูลเจาะจง เช่น คำสั่งใน SNMP
  - 11 = ประเภท Private ข้อมูลสำหรับใช้กรณีเฉพาะ
2. บิตถัดมาคือฟิลด์ form กำหนดว่าแบบข้อมูลนั้นเป็นแบบ พื้นฐาน(Primitive) หรือ แบบโครงสร้าง(Constructed) เช่น ตัวอย่างแบบข้อมูลพื้นฐาน เช่น integer หรือ string ส่วนข้อมูลแบบโครงสร้าง ก็เช่น sequence
  3. Tag number 5 bits สุดท้าย : เป็นส่วนกำหนดแบบข้อมูลซึ่งมีค่าได้ตั้งแต่ 0 ถึง 30

|                    | แบบข้อมูล        | value (ฐาน 16) |
|--------------------|------------------|----------------|
| Universal          | Integer          | 02             |
|                    | OctetString      | 04             |
|                    | null             | 05             |
|                    | objectIdentifier | 06             |
|                    | sequence         | 16             |
| Application - wide | IPAddress        | 40             |
|                    | Counter          | 41             |
|                    | Gauge            | 42             |
|                    | TimeTicks        | 43             |
|                    |                  |                |
| Context - specific | get-request      | A0             |
|                    | get-next-request | A1             |
|                    | get-response     | A2             |
|                    | set-request      | A3             |
|                    | trap             | A4             |

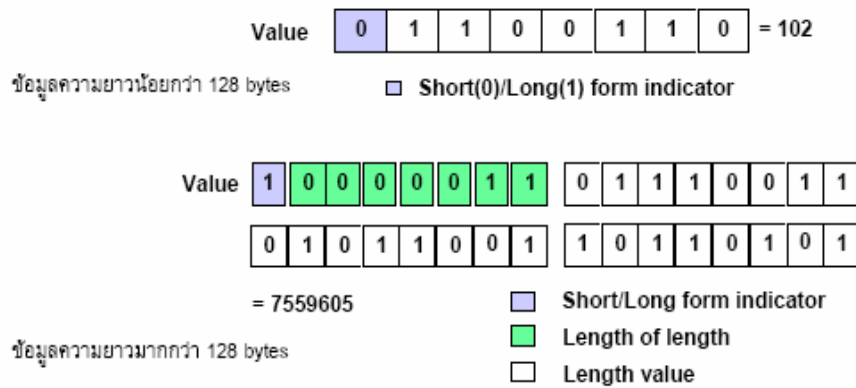
**รูปที่ 2.14** รูปแบบข้อมูลที่ใช้ใน SNMP

ตัวอย่างเช่น ข้อมูล Integer จะมี type เท่ากับ 02 หรือแยกออกมาเป็น bit ได้เป็น 0000 0010 ซึ่งมาจาก 00(Universal) , 0(Primitive) และ 0 0010(Tag Number)

#### ฟิลด์กำหนดความยาว (Length)

หากข้อมูลความยาวน้อยกว่า 128 bytes ฟิลด์นี้จะกินเนื้อที่เพียง 1 byte โดยมี bit ซ้ายสุดเป็น “0” และ 7 bit ที่เหลือกำหนดความยาว เช่น ข้อมูลยาว 10 byte ค่าในฟิลด์จะเท่ากับ 0x0A

หากข้อมูลมีความยาวตั้งแต่ 128 bytes ขึ้นไป ต้องใช้ฟิลด์ length หลายออกเทต โดยที่บิตซ้ายสุดจะมีค่าเป็น “1” และใช้ 7 bits ที่เหลือเป็นตัวนับจำนวนออกเทตที่กำหนดความยาว ถัดจากนั้นจึงเป็นไบต์กำหนดความยาว เช่น ข้อมูลยาว 1000 bytes (03 E8) ค่าในฟิลด์นี้จะเป็น 82 03 E8 โดยที่ 7 bits ขวาของ B2 คือ 000 0010 ซึ่งเท่ากับ 2 หมายถึงใช้ 2 bytes กำหนดความยาว และ 2 bytes นั้นคือ 03 E8



รูปที่ 2.15 แสดงตัวอย่างการเข้ารหัสความยาว

### ฟิลด์กำหนดข้อมูล (Value)

การเข้ารหัสฟิลด์ content จะแตกต่างกันไปตามชนิดของข้อมูล ดังนี้

**รูปแบบการเข้ารหัส TLV** (เฉพาะบางประเภทข้อมูลที่ใช้ใน SNMP)

1. ข้อมูลประเภทสายอักขระ ไม่ต้องเข้ารหัส ส่งไปตามลำดับของสายอักขระตามปกติ เช่น สายอักขระ “interfaces” จะอยู่ในรหัส TLV ดังนี้ 04 0A ‘i’ ‘n’ ‘t’ ‘e’ ‘r’ ‘f’ ‘a’ ‘c’ ‘e’ ‘s’ 04 คือ OctetString และ 0A คือความยาว
2. ข้อมูลตัวเลข
  - ค่าไม่เกิน 127 จะใช้เพียง 1 byte เท่านั้น
  - ค่าเกิน 127 จะต้องผ่านการเข้ารหัสอีกที คือ set bit ซ้ายสุดให้เป็น 1 และใช้ 7 bit ที่เหลือกำหนดจำนวน octet ที่ต้องใช้ จากนั้นจึงค่อยตามด้วยค่า octet ถัดต่อไป เช่น ค่าตัวเลข 130 เมื่อเข้ารหัสแล้วจะได้ค่า 02 02 81 02 โดยที่ 02 คือ integer และ 81 02 แทนค่า 130
3. Null ให้ส่งโดยเพียงแต่กำหนดค่าในฟิลด์ length เป็น 0 และไม่ต้องส่งค่าใดๆไปทั้งสิ้น
4. ข้อมูลประเภท objectIdentifier ต้องเข้ารหัสด้วยวิธีพิเศษ

### ตัวอย่าง SNMP Frame และการเข้ารหัส

#### การส่งคำสั่ง request (GetRequest PDU)

ลำดับ Byte เมื่อใช้คำสั่ง GetRequest PDU สอบถามค่า 1.3.6.1.2.1.1.0 หรือ sysDescr.0 แต่ละฟิลด์ของ SNMP จะผ่านการเข้ารหัสตามโครงสร้าง TLV

|     |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|----|----|----|----|----|----|----|----|
| UDP | 30 | 27 | 02 | 01 | 00 | 04 | 06 | 73 | 23 | 6E |
|     | 6D | 70 | 21 | A0 | 1A | 02 | 02 | 22 | FB | 02 |
|     | 01 | 00 | 02 | 01 | 00 | 30 | 0E | 30 | 0C | 06 |
|     | 08 | 2B | 06 | 01 | 02 | 01 | 01 | 01 | 00 | 05 |
|     | 00 |    |    |    |    |    |    |    |    |    |

รูปที่ 2.16 SNMP Frame ของ GetRequest PDU

|    |    |    |    |    |    |    |    |    |    |  |
|----|----|----|----|----|----|----|----|----|----|--|
| 30 | 27 |    |    |    |    |    |    |    |    |  |
| 02 | 01 | 00 |    |    |    |    |    |    |    |  |
| 04 | 06 | 73 | 23 | 6E | 6D | 70 | 21 |    |    |  |
| A0 | 1A |    |    |    |    |    |    |    |    |  |
| 02 | 02 | 22 | FB |    |    |    |    |    |    |  |
| 02 | 01 | 00 |    |    |    |    |    |    |    |  |
| 02 | 01 | 00 |    |    |    |    |    |    |    |  |
| 30 | 0E |    |    |    |    |    |    |    |    |  |
| 30 | 0C |    |    |    |    |    |    |    |    |  |
| 06 | 08 | 2B | 06 | 01 | 02 | 01 | 01 | 01 | 00 |  |
| 05 | 00 |    |    |    |    |    |    |    |    |  |

| Type         | Length | value      | หมายเหตุ                |
|--------------|--------|------------|-------------------------|
| sequence     | 39     | -          | มีข้อมูล 39 bytes ตามมา |
| integer      | 1      | 0          | version = 1             |
| string       | 6      | #snmp!     | community = #snmp!      |
| context-spc. | 26     | -          | get request 26 bytes    |
| integer      | 2      | 22FB       | id = 22FB               |
| integer      | 1      | 0          | error status = 0        |
| integer      | 1      | 0          | error status = 0        |
| sequence     | 14     | -          |                         |
| sequence     | 12     | -          |                         |
| oid          | 8      | sysDescr.0 | 1.3.6.1.2.1.1.1.0       |
| null         | 0      | 0          | รหัสปิดท้าย             |

รูปที่ 2.17 ความหมายของการเข้ารหัสข้อมูลของ GetRequest PDU

#### การตอบกลับคำสั่ง request (GetResponse PDU)

ลำดับ byte เมื่อ agent ใช้คำสั่ง GetResponse PDU ตอบค่า sysDescr.0 ส่งกลับไป แต่ฟิลด์ที่ผ่านการเข้ารหัสตามโครงสร้าง TLV จะแสดงได้ดังนี้ (เนื่องจากสายอักขระที่ตอบกลับมีความยาวมากกว่า 300 byte ดังนั้นจึงเลือกแสดงแค่บางส่วนเท่านั้น)

|     |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|----|----|----|----|----|----|----|----|
| UDP | 30 | 81 | F9 | 02 | 01 | 00 | 04 | 06 | 73 | 23 |
|     | 6E | 6D | 70 | 21 | A2 | 81 | EB | 02 | 02 | 02 |
|     | FB | 02 | 01 | 00 | 02 | 01 | 00 | 30 | 81 | DE |
|     | 30 | 81 | DB | 06 | 08 | 2B | 06 | 01 | 02 | 01 |
|     | 01 | 01 | 04 | 81 | CE | 43 | 65 | 73 | 63 | 6F |

รูปที่ 2.18 SNMP Frame ของ GetResponse PDU

| Type         | Length | value      | หมายเหตุ                 |
|--------------|--------|------------|--------------------------|
| sequence     | 377    | -          | มีข้อมูล 377 bytes ตามมา |
| integer      | 1      | 0          | version = 1              |
| string       | 6      | #snmp!     | community = #snmp!       |
| context-spc. | 363    | -          | get response 363 bytes   |
| integer      | 2      | 22FB       | id = 22FB                |
| integer      | 1      | 0          | error status = 0         |
| integer      | 1      | 0          | error index = 0          |
| sequence     | 14     | -          |                          |
| sequence     | 12     | -          |                          |
| oid          | 8      | sysDescr.0 | 1.3.6.1.2.1.1.1.0        |
| OctectString | 334    | -          |                          |
|              |        |            | Cisco.....               |

รูปที่ 2.19 ความหมายของการเข้ารหัสข้อมูลของ GetResponse PDU